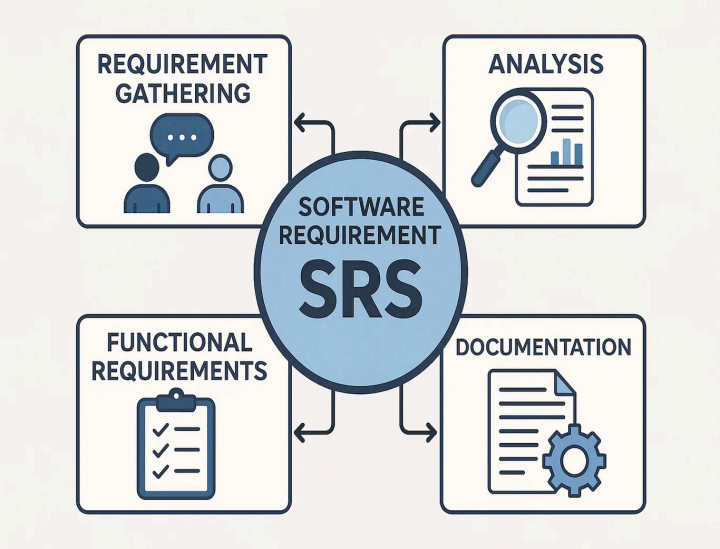
Team 4

Software Requirements Specification (SRS)

Terrapin Events



CLIN:	001
Version #:	2.0
Version Date:	10/25/2025
Team Name:	Team 4
Lead Author:	Bhavna Kumari
Contributing Author(s):	Abhigna Kandala, Heena Khan, Teja Tatimatla, Vineet Agarwal

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Contents

Table of Contents **1**
Table of Figures **3**
Table of Charts **4**
1. Introduction **5**
1.1. Document Purpose 5
1.2. Project Description / System Scope 6
Elicitation Phase (Extra Credit Attempted) 6
Feature Tree for Terrapin Events (Extra Credit Attempted) 7
1.3. Intended Audience and Usage 9
2. Overall Description **10**
Analysis Phase (Extra Credit Attempted) 10
2.1. Product Perspective 10
New System 10
Upgraded Components 10
Integrated System 10
Work Context Diagram Description (Extra Credit Attempted) 11
2.2. Product Functions 12
2.3. User Classes and Characteristics 13
2.4. Operating Environment 13
2.5. Design and Implementation Constraints” 14
2.6. Assumptions and Dependencies 14
3. System Requirements **16**
Specification Phase (Extra Credit Attempted) 16
3.1. Functional Requirements 16
Functional Requirements Prioritization - MOSCOW Method (Extra Credit Attempted) 18
3.2. Non-Function Requirements (NFRs) 19
Table : Quality Attribute Scenario (SEI Template – Performance) (Extra Credit Attempted) 20
Non-Functional Requirements Prioritization : Analytical Hierarchy Process (AHP) (Extra Credit Attempted) 20
3.3. Abuse Cases (Security and Misuse Scenarios) 21
Abuse Case Diagram for Unauthorized Role Escalation (Extra Credit Attempted) 22
Abuse Case Diagram for Event Data Manipulation by Unauthorized User (Extra Credit Attempted) 23
Summary of Threat Coverage (Extra Credit Attempted) 25
3.4. Requirements Traceability Matrix 25
Requirements Traceability Matrix Table (Extra Credit Attempted) 26
4. External Interfaces Requirements **27**
Validation Phase (Extra Credit Attempted) 27
4.1. User Interfaces 27

Copyright © 2025 Tony D Barber. All rights reserved.

4.2. Software Interfaces....34
4.3. Hardware Interfaces....34
4.4. Communication Interfaces....34
5. Use Cases Diagrams and Narrative / Scenarios....36
UC-01: Sign in with CAS SSO....36
UC-02 Create and Submit Event for Approval....37
UC-03 Approve Event....39
UC-04 Discover and Register for Event....40
UC-05 Pay for Ticketed Event....42
6. Quality Attribute Scenarios....45
QAC - 001 - Performance....45
QAC - 002 - Availability & Reliability....45
QAC - 003 - Security & Compliance....46
QAC - 004 - Usability & Accessibility....47
QAC - 005 - Scalability....47
QAC - 006 - Maintainability & Deployability....48
QAC - 007 - Data Integrity....49
7. Utility Tree....50
Summary of Quality Attributes and Architecturally Significant Requirements (ASRs) (Extra Credit Attempted)....52
8. System Models....53
Class Diagram....53
Sequence Diagram....54
Activity Diagrams....55
AD-01 UC-01: Sign in with CAS SSO....55
Activity Diagram-01 Legend Description....56
AD-02 UC-02: Create and Submit an Event for Approval....57
Activity Diagram-02 Legend Description....58
Software Requirement Specification Phase (Extra Credit Attempted)....58
Agile Update (Extra Credit Attempted)....59
Progress Tracker Update (Extra Credit Attempted)....60
Appendix A....61
Appendix B....62
Appendix C....63

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Figures

Figure 1: Software Development Lifecycle..... 5
Figure 2: Elicitation Phase..... 6
Figure 3 - An example of grouping of a feature and its sub features..... 8
Figure 4 - Feature Tree for Terrapin Events..... 9
Figure 5: Analysis Phase..... 10
Figure 6 : Work Context Diagram..... 11
Figure 7: Specification Phase..... 16
Figure 8 : Types of Non Functional Requirements..... 19
Figure 9 : AHP Prioritization for Non-Functional Requirements..... 21
Figure 10 : Abuse Case - 01..... 22
Figure 11 : Abuse Case - 02..... 23
Figure 12 : Validation Phase..... 27
Figure 13 : Landing Page..... 28
Figure 14: Authorized User - Home..... 29
Figure 16: Authorized User - Event Detail..... 29
Figure 17 : Organizer - Home..... 31
Figure 18 : Organizer - Event Detail..... 31
Figure 19 : Organizer - Multi-step Wizard..... 31
Figure 20: Admin - Home..... 33
Figure 21: Admin - Event Detail..... 33
Figure 22: Branding Sample..... 33
Figure 23: Use Case 001 - Sign with CAS SSO..... 37
Figure 24: Use Case - 002- Create and Submit Event for Approval..... 39
Figure 25: Use Case - 003- Approve Event..... 40
Figure 26: Use Case - 004- Discover and Register for Event..... 42
Figure 27: Use Case - 005- Pay for Ticketed Event..... 44
Figure 28 : Utility Tree..... 50
Figure 29: Quality Attribute Scenario Prioritization..... 52
Figure 30: Class Diagram of Terrapin Events..... 53
Figure 31: Sequence Diagram for Terrapin Events..... 54
Figure 32: Activity Diagram for Use Case - 001- Sign with CAS SSO..... 55
Figure 33: Activity Diagram for Use Case - 002- Create and Submit Event for Approval..... 58
Figure 34: Software Requirement Specification..... 59
Figure 35: Sprint 2..... 59
Figure 36: Progress Tracker..... 60

Copyright © 2025 Tony D Barber. All rights reserved.

Table of Charts

Table 1 - Audience and Usage purposes..... 9
Table 2: Entity Role Mapping..... 12
Table 3: User Classes and Characteristics..... 13
Table 4: Assumptions and Dependencies..... 15
Table 5 : Functional Requirement Prioritization Method..... 19
Table 6 : Performance Quality Attribute Scenario..... 20
Table 7 : Summary of Threat Coverage..... 25
Table 8 : Legends Description of AD-001..... 55
Table 9 : Legends Description of AD-002..... 57

Copyright © 2025 Tony D Barber. All rights reserved.

1. Introduction

1.1. Document Purpose

This **Software Requirements Specification (SRS)** formally defines all **functional**, **non-functional**, and **security** requirements for the **Terrapin Events** System, a Campus Event Management System (CEMS) developed by **Team 4** at the **University of Maryland, College Park**.

It establishes the **baseline reference** for all design, development, and testing activities, ensuring every implemented feature can be traced back to a validated requirement.

Purpose within Lifecycle

This SRS functions as the **formal requirements baseline** that connects the Project Management Plan to all subsequent development phases.

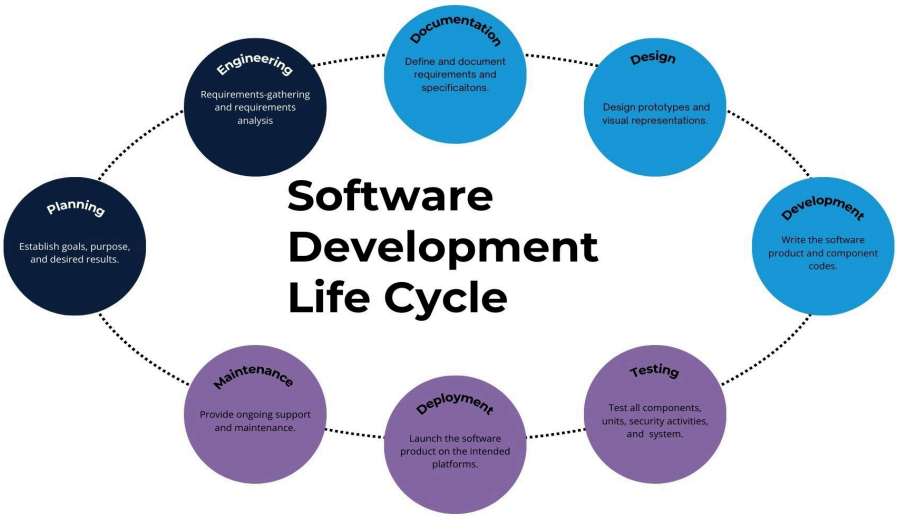


Figure 1: Software Development Lifecycle

It will directly guide the following phases:

- **Architecture design:** Translating functional and non-functional requirements into components and deployment models.
- **UI prototype development:** Ensuring user interactions align with defined use cases and access roles.
- **Testing and verification:** Providing measurable, traceable inputs for acceptance and regression testing.
- **Change Control & Risk Tracking:** Establishing measurable requirements that can be version-controlled as stakeholder feedback evolves.

Copyright © 2025 Tony D Barber. All rights reserved.

Throughout the lifecycle, this document will also support **change management and risk tracking** by providing measurable, traceable and testable requirements that can be updated or version-controlled as stakeholder feedback evolves.

1.2. Project Description / System Scope

Elicitation Phase (Extra Credit Attempted)

- Collecting the user's requirements

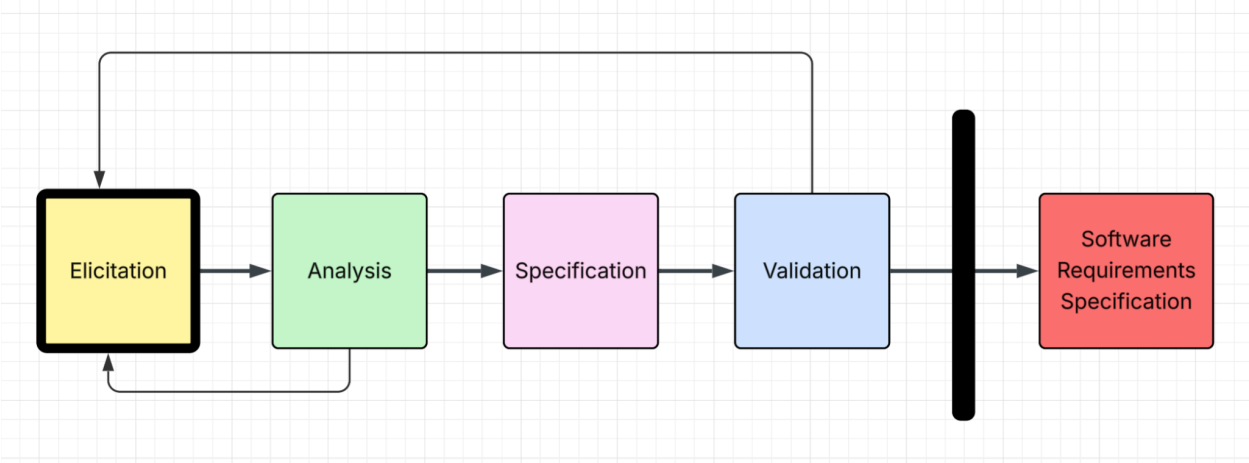


Figure 2: Elicitation Phase

The **Terrapin Events** system provides a **centralized digital platform** for **planning, organizing, registering, and tracking** campus events at the University of Maryland.
It replaces the current manual and fragmented methods (e.g., emails, flyers, department sites) with a unified solution that enhances accessibility, efficiency, and engagement across students, faculty, and administrators.

System Objectives

1. **Provide a centralized digital platform** for creating, managing, and promoting campus events.
2. **Enable secure user authentication** through UMD CAS Single Sign-On (SSO) and implement role-based access control for students, organizers, administrators, and IT staff.
3. **Allow participants to register for events**, manage cancellations or waitlists, and receive confirmation or reminder notifications.
4. **Support payment processing** for ticketed or premium events through a secure, university-approved payment gateway.
5. **Facilitate post-event feedback collection** and maintain a transparent event history for organizers and administrators.
6. **Ensure accessibility, data privacy, and reliability** in compliance with FERPA, PCI DSS, and ADA standards.

System Boundaries

Copyright © 2025 Tony D Barber. All rights reserved.

- **In-Scope:** Web-based portal with authentication via UMD CAS, event management modules, calendar integration, notifications, and reporting features.
- **Out-of-Scope:** Native mobile applications, third-party financial system integrations beyond the approved gateway, and long-term post-deployment maintenance.

Context and Purpose within Lifecycle

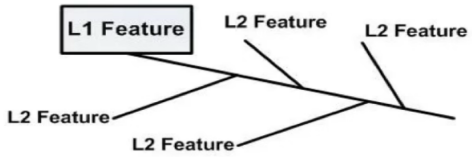
Terrapin Events addresses the long-standing challenge of fragmented event management processes at the University of Maryland. By replacing manual workflows and disjointed communication channels, it introduces an efficient, transparent, and secure event-management ecosystem that benefits both organizers and attendees.

Within the project lifecycle, this system represents the Requirements Analysis Phase deliverable under ENPM 613. It bridges the transition from conceptual planning (PA 1 – PMP) to technical design (PA 3 – Architecture & Detailed Design). This ensures that every architectural and implementation decision is traceable to stakeholder-approved requirements, fulfilling the core principle of *requirements engineering* emphasized in the course rubric.

Feature Tree for Terrapin Events (Extra Credit Attempted)

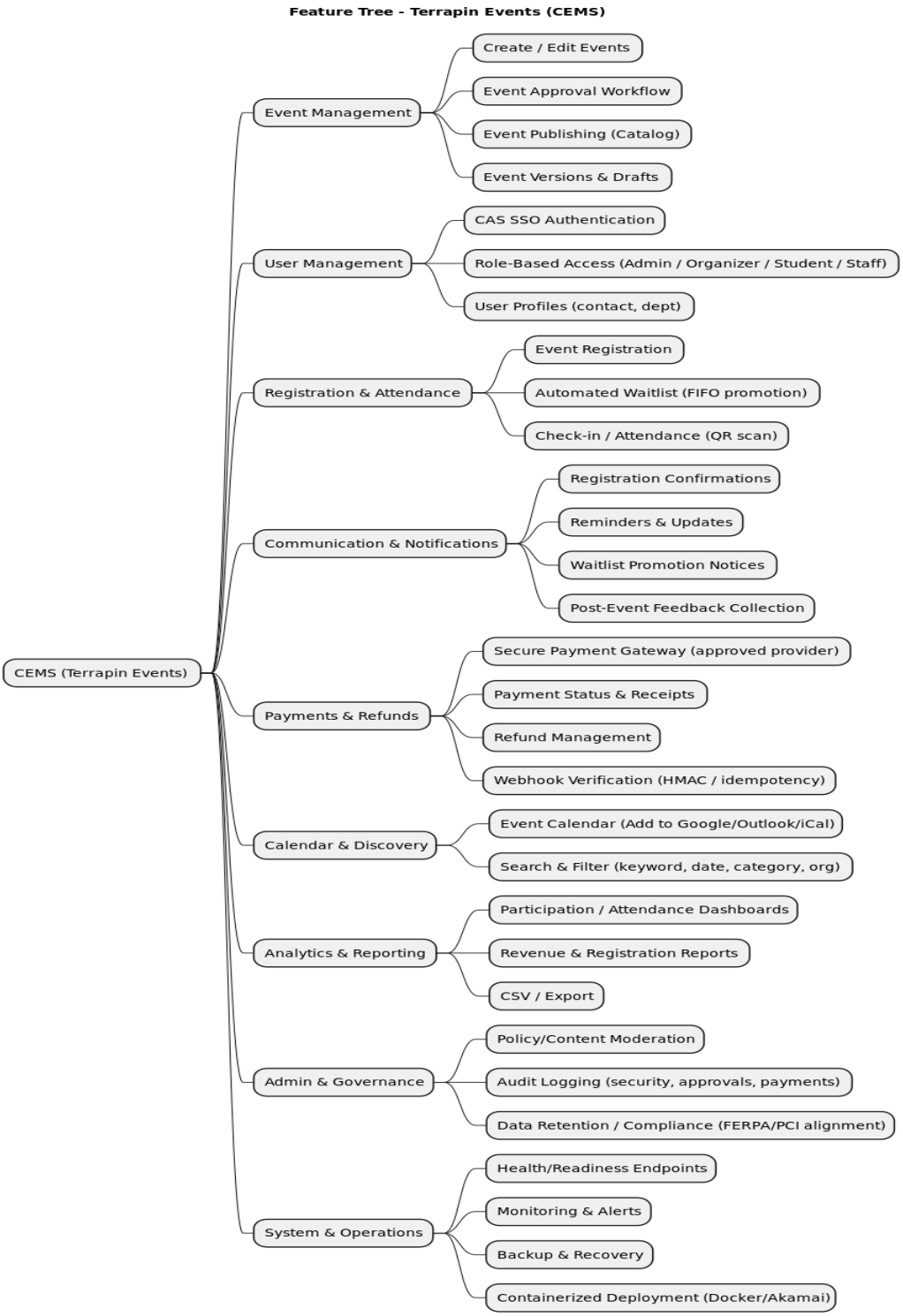
A visual model for organising a set of features:

- A feature consists of one or more logically related system capabilities that provide and are described by a set of functional requirements
- Used for understanding related features (and sub-features) and assisting with them to development iterations or product releases



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 3 - An example of grouping of a feature and its sub features



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 4 - Feature Tree for Terrapin Events

Link to view the diagram : Feature Tree Terrapin Events

1.3. Intended Audience and Usage

Table 1 - Audience and Usage purposes

Audience	Purpose of Use
Developers & Architects	Design and implement system modules consistent with the defined requirements.
Test Engineers & QA Team	Derive test cases and verification procedures for each FR, NFR, and abuse case.
Project Manager & Faculty Advisor	Track scope, quality, and traceability across deliverables and sprints.
Stakeholders	Validate that system behavior meets business needs, policy requirements, and compliance standards.

Copyright © 2025 Tony D Barber. All rights reserved.

2. Overall Description

Analysis Phase (Extra Credit Attempted)

- Understanding and modelling the required behaviour.

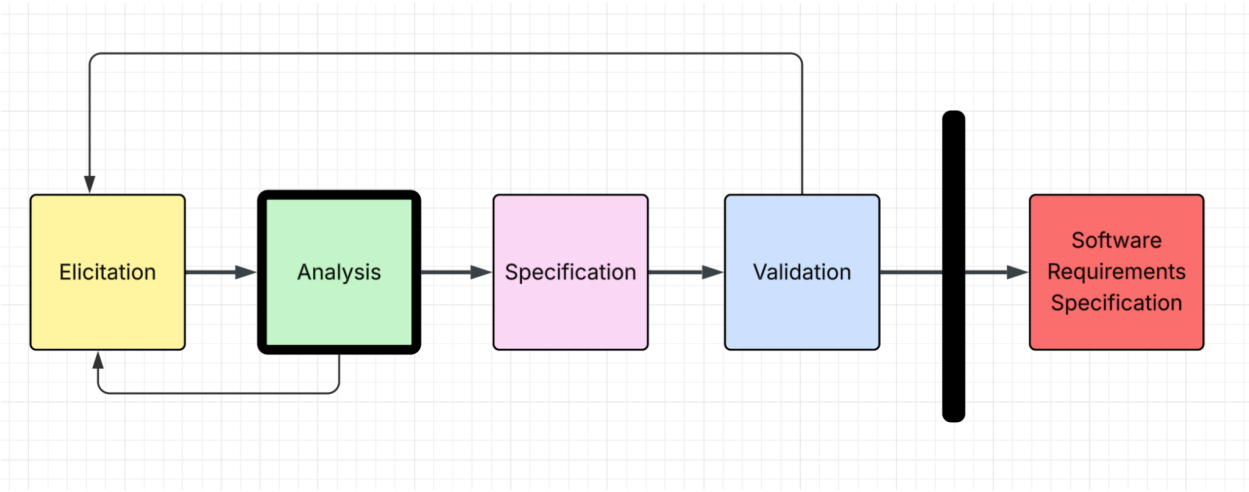


Figure 5: Analysis Phase

2.1. Product Perspective

New System

Terrapin Events is a new web-based application developed as part of the University's Campus Event Management System (CEMS) ecosystem.

It provides a centralized platform for creating, managing, and registering campus events, features that previously did not exist in a single unified system.

Upgraded Components

While primarily a new system, **Terrapin Events** incorporates several enhanced functionalities such as automated waitlisting, QR-based attendance tracking, and real-time notifications.
These components represent process upgrades over traditional manual workflows (e.g., paper registration forms or scattered department-level tools).

Integrated System

The system is designed to integrate seamlessly with existing university infrastructure, including:

- UMD CAS (Single Sign-On): for user authentication and secure access control.
- SMTP Email Services: for automated notifications and communication.
- University-Approved Payment Gateway: for secure online payment processing and transaction validation.

Copyright © 2025 Tony D Barber. All rights reserved.

Terrapin Events operates as a **stand-alone, containerized application** hosted on university-approved servers and databases, serving as the **authoritative source for event scheduling, registration, and reporting** within the CEMS framework.

Work Context Diagram Description (Extra Credit Attempted)

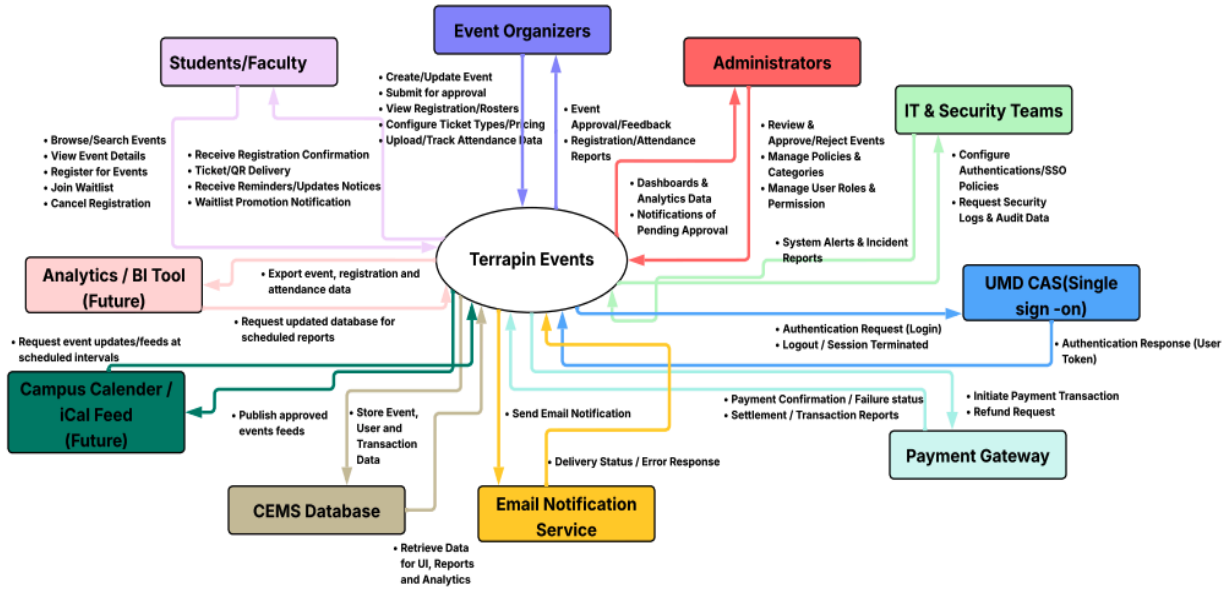


Figure 6 : Work Context Diagram

Work Context Diagram Description

The **Work Context Diagram** illustrates how the **Terrapin Events System** interacts with both internal and external entities within the **University of Maryland** environment.
It defines the system's **operational boundaries**, showing the external actors and services that exchange data with the system through defined interfaces such as APIs and authentication protocols.

Key interactions include:

- **User access and authentication** through the university's CAS Single Sign-On (SSO) service.
- **Event creation, registration, and attendance tracking** performed by users with different roles.
- **Secure communication** with external systems like the **Payment Gateway**, **Email Notification Service**, and **CEMS Database**.
- **Future integration points** for analytics tools and campus calendar feeds to extend data visibility and reporting.

This diagram establishes the **system boundary**, identifies trusted versus external components, and clarifies how information flows between users, institutional infrastructure, and supporting services.

Work Context Entity, Roles and Interaction Segregation

Copyright © 2025 Tony D Barber. All rights reserved.

Entity	Role and Interaction with Terrapin Events
Students and Faculty	Interact with the system via the web portal to browse, register, and check into events. Data exchanged: registration requests, confirmations, feedback.
Event Organizers	Interact with the application to create and manage events; submit event data and retrieve registration and attendance reports.
Administrators	Access the admin dashboard to approve events, manage permissions, and monitor logs..
IT and Security Teams	Maintain operational security by managing audit logs, monitoring system health endpoints, and enforcing access policies.
UMD CAS (SSO)	Provides authentication tokens to the **Terrapin Events** login module using the CAS protocol for secure identity verification.
Payment Gateway	Handles HTTPS-based transaction flows for ticketed events, returning authorization responses and payment status.
Email Notification Service	Receives outbound SMTP or API calls to deliver registration confirmations, reminders, and waitlist notifications.
CEMS Database	Acts as the primary persistent data store for user, event, registration, and transaction entities accessed via RESTful APIs.
Campus Calendar Feed (Future)	Consumes published event data through iCal/ICS endpoints for department-wide visibility.
Analytics / BI Tool (Future)	Connects through read-only APIs to retrieve aggregated event and participation data for reporting.

Table 2: Entity Role Mapping

2.2. Product Functions

At a high level, Terrapin Events will:

- **Authenticate users** through UMD CAS SSO and establish secure sessions.
- **Enforce role-based permissions (RBAC)** for Administrators, Event Organizers, Students/Faculty, and IT staff.
- **Allow event organizers** to create, edit, and submit events for administrator approval.
- **Enable administrators** to approve, reject, or modify event submissions before publication.

Copyright © 2025 Tony D Barber. All rights reserved.

- **Publish approved events** to a centralized, searchable event catalog with filtering by category, organizer, date, and location.
- **Allow students and faculty** to register for events, join waitlists when capacity is reached, and receive automatic confirmations or updates.
- **Integrate secure online payments** for ticketed events using the approved gateway.
- **Provide notifications and reminders** via email and in-app messages for registrations, cancellations, and waitlist promotions.
- **Support feedback collection** post-event to inform organizers of user satisfaction and potential improvements.
- **Maintain system integrity and compliance** through audit logging, session security, and data-protection controls.

2.3. User Classes and Characteristics

User Class	Description/Role	Technical Expertise
Students and Faculty	Discover and register for events, receive notifications, and submit feedback.	Basic computer skills
Event Organizers	Create and manage events, track attendance, and communicate with participants.	Intermediate
Administrators	Approve events, manage roles, monitor analytics and policy compliance.	Intermediate
IT and Security Teams	Maintain servers, monitor logs, and ensure compliance (FERPA, PCI DSS).	Expert

Table 3: User Classes and Characteristics

2.4. Operating Environment

- **Platform:** Web-based, responsive application accessible through modern browsers (Chrome, Firefox, Edge, Safari).
- **Supported Operating Systems:** Windows, macOS, and Linux.
- **Server Environment:** Containerized via Docker with FastAPI backend and MongoDB database.
- **CI/CD Pipeline:** Continuous integration and deployment through GitHub Actions with automated testing and rollback support.
- **Cloud Infrastructure:** Hosted on an approved cloud provider Linode to ensure high availability and scalability.
- **Authentication:** Single Sign-On via UMD CAS for secure identity verification.
- **Network & Security:** Encrypted HTTPS (TLS 1.3), VPN access for administrative tasks, and enforced HSTS.

Copyright © 2025 Tony D Barber. All rights reserved.

- **Accessibility:** Compliant with ***WCAG*** **2.1 AA** and **ADA** standards, including keyboard navigation, color-contrast, and screen-reader support.

2.5. Design and Implementation Constraints”

- Compliance with ***FERPA*** (for protection of student data) and ***PCI DSS*** (for secure handling of payment information).
- Adherence to **University IT security policies**, including data retention, logging, and periodic audits.
- Deployment must use **containerization (Docker)** to ensure environment consistency across development and production.
- Source code must be maintained in a **university-approved Git repository** with access control and versioning.
- Implementation is limited to **open-source or university-licensed frameworks** (React / FastAPI / MongoDB stack).
- All environment variables and secrets shall be stored in encrypted form in GitHub Secrets or cloud key vaults.
- **Integration testing** must be performed in a controlled staging environment before production rollout.
- **Timeline Constraint:** All requirements, coding, and validation activities must be completed within the semester timeframe (Weeks 5–14).
- **Operational Constraint:** The application will not include native mobile apps or external third-party APIs not approved by UMD.
- **Performance Constraint:** Response time for all primary user operations (login, search, registration) must not exceed 2 seconds for 95 % of requests under normal load.

2.6. Assumptions and Dependencies

Assumption / Dependency	Impact on Project
User Authentication	All users (students, faculty, organizers, admins) possess valid UMD Directory IDs and access to CAS authentication. → Critical for secure access control and role enforcement.
Infrastructure Availability	University servers and cloud resources remain operational throughout the semester. → Any prolonged downtime directly affects testing and deployment schedules.
Network Connectivity	Continuous internet and campus VPN access are assumed for both users and developers. → Network interruptions could delay real-time notification and payment modules.
Third-Party Services	The system depends on university-approved SMTP email service and a secure payment gateway with high availability. → Service failures would impact critical notifications and transaction flows.

Copyright © 2025 Tony D Barber. All rights reserved.

Technology Stack Stability	Node.js, FastAPI, and MongoDB versions used must remain stable throughout development. → Unexpected version updates may affect deployment consistency.
Browser Compatibility	Application will be tested and optimized for latest versions of Chrome, Firefox, Edge, and Safari. → Older browsers may experience reduced functionality.
Data Security and Compliance	All team members will follow UMD's data-handling guidelines for FERPA and PCI DSS compliance. → Non-adherence could result in policy violations and data breach risks.
Project Timeline	The project must adhere to the academic schedule with deliverables due weekly per PA milestones. → Delays in earlier phases may compress testing time and impact final delivery.
Team Coordination Tools	Collaboration relies on Jira and GitHub for task tracking and source control. → Tool downtime can temporarily disrupt progress monitoring and reporting.

Table 4: Assumptions and Dependencies

Copyright © 2025 Tony D Barber. All rights reserved.

3. System Requirements

Specification Phase (Extra Credit Attempted)

- Documenting the behaviour of the proposed system

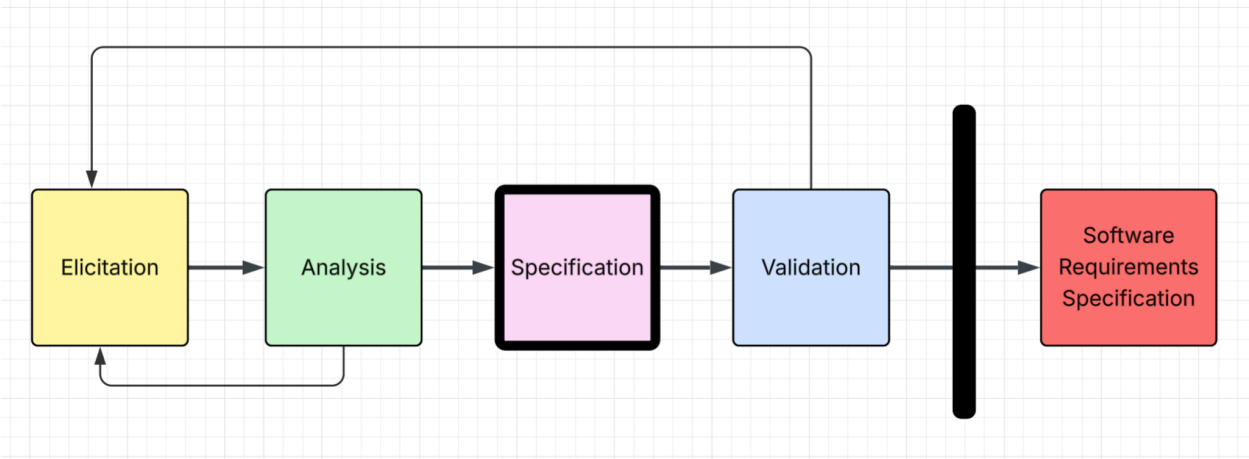


Figure 7: Specification Phase

3.1. Functional Requirements

- **FR-001 Authentication**
  The system shall authenticate all users through the University of Maryland *CAS*.
- **FR-002 Application Session**
  The system shall initiate an application session upon successful login.
- **FR-003 Role-Based Access Control** *RBAC*
  The system shall enforce *RBAC* for the *Authorised Roles*. Each role shall have predefined permissions and restricted access to ensure data security and compliance.
- **FR-004 User Profile View**
  The system shall allow users to view their *Profile Information* used for event communication.
- **FR-005 User Profile Update**
  The system shall allow users to update their *Profile Information* used for event communication.
- **FR-006 Event Creation**
  The system shall allow Event Organizers to create new events with *Event Details*.
- **FR-007 Event Editing**
  The system shall allow Event Organizers to edit draft events with *Event Details*.
- **FR-008 Event Versioning**
  The system shall maintain a version history for all published events.
- **FR-009 Event Review Workflow**
  WHEN a new event is created, the system shall require the event to be reviewed by an Administrator before it is published on the platform.
  ***** Type of Requirement:*** *Event Driven Requirement*
- **FR-010 Event Approval Workflow**
  WHEN a new event has been reviewed, the system shall require the event to be approved by an Administrator before it is published on the platform.

Copyright © 2025 Tony D Barber. All rights reserved.

**** Type of Requirement:*** *Event Driven Requirement*

- **FR-011 Event Publishing**
  The system shall publish approved events, making them searchable and visible to Students and Staff.
- **FR-012 Search and Discovery**
  The system shall allow users to search and filter events by *Event Details*.
- **FR-013 Registration**
  The system shall allow *Authorised Users* to register for events with available capacity.
- **FR-014 Automated Waitlist**
  IF an event reaches its maximum capacity, THEN the system shall automatically place any new registrants on a waitlist and promote users in a first-come, first-served order when seats become available, notifying them accordingly.
  **** Type of Requirement:*** *Unwanted Behavior*
- **FR-015 Waitlist Promotion**
  IF seats become available in a full event, THEN the system shall promote users from the waitlist in a first-come, first-served order and notify them accordingly.
  **** Type of Requirement:*** *Unwanted Behavior*
- **FR-016 Check-in and Attendance Tracking**
  The system shall allow Event Organizers and Staff to mark attendance or scan QR codes at event check-in points.
- **FR-017 Payment Integration for Paid Events**
  The system shall support online payments through a secure, university-approved payment gateway for paid events and record the payment status.
- **FR-018 Refund Management**
  The system shall allow Administrators and Event Organizers to process refund requests and record refund outcomes.
- **FR-019 Notifications**
  The system shall send automated notifications to *Authorised Users* regarding *Event Related Notifications*.
- **FR-020 Calendar Integration**
  The system shall provide calendar integration features, including “Add to Calendar” options *Calendar Types* for registered users.
- **FR-021 Feedback Collection**
  The system shall allow *Authorised Users* to submit feedback after attending an event.
- **FR-022 Feedback Analytics**
  Event Organizers and Administrators shall be able to view summarized feedback analytics.
- **FR-023 Reporting and Analytics**
  The system shall allow Administrators and Staff to generate reports related to *Event Insights*.
- **FR-024 Audit Logging**
  The system shall maintain audit logs for sensitive actions, including *System Functionalities*.
- **FR-025 System Health and Monitoring**
  The system shall provide health endpoints and performance logs for Administrators to monitor *Application Monitoring*.
- **FR-026 Role-Based Dashboard Access**

Copyright © 2025 Tony D Barber. All rights reserved.

The system shall provide customized dashboards for each *Role-Based Dashboard Access*. Each dashboard shall display only the data relevant to the user's role.

Functional Requirements Prioritization - MOSCOW Method (Extra Credit Attempted)

- **Moscow Method Description -** The MoSCoW Method is a prioritization technique that categorizes requirements into Must Have, Should Have, Could Have, and Won't Have to clarify delivery priorities.
- It helps teams focus on critical features first while managing scope and expectations effectively.

Requirement ID	Requirement Description	Priority	Justification
FR-001	CAS SSO authentication for all users	Must Have	Core requirement for secure university login
FR-002	Role-based access control for Admin, Organizer, Student, Staff	Must Have	Ensures data security and controlled access
FR-004	Event creation by organizers	Must Have	Core system function supporting primary stakeholder need
FR-006	Administrator approval workflow	Must Have	Maintains event integrity and moderation
FR-009	Event registration by Students/Staff	Must Have	Essential functionality for system adoption
FR-010	Automated waitlist management	Should Have	Improves efficiency and user satisfaction
FR-012	Payment integration for paid events	Should Have	Required for monetized university events
FR-014	Email/SMS notifications	Should Have	Enhances communication and reduces no-shows
FR-016	Post-event feedback form	Could Have	Useful for event improvement; not core for MVP
FR-020	Role-based dashboards	Should Have	Improves usability and analytics for different user roles

Copyright © 2025 Tony D Barber. All rights reserved.

Table 5 : Functional Requirement Prioritization Method

3.2. Non-Function Requirements (NFRs)

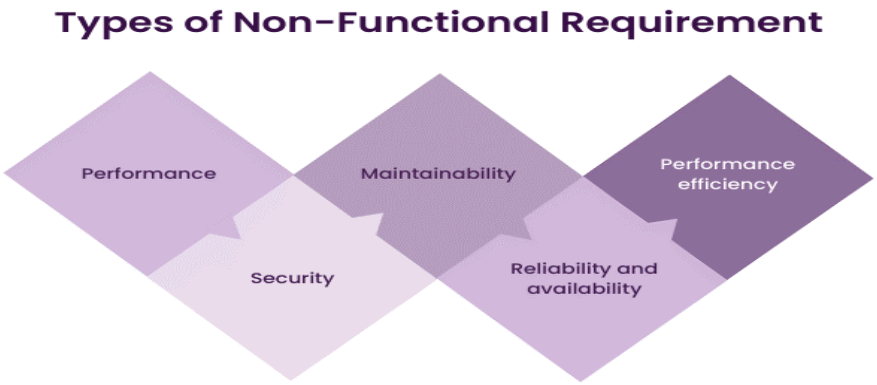


Figure 8 : Types of Non Functional Requirements

- **NFR-001 Performance**
  The system shall respond to all *User-Initiated Operations* within 800 milliseconds for 99.999 % of requests under normal load.
- **NFR-002 Scalability**
  The system shall support *Horizontal Scaling* using containerized deployments on Akamai Connected Cloud to handle 2× baseline concurrent traffic without service degradation.
- **NFR-003 Availability**
  The system shall maintain ≥ 99.99 % uptime per month, excluding scheduled maintenance windows.
- **NFR-004 Security – Authentication and Authorization**
  All user sessions shall be authenticated through CAS SSO and protected via JWT tokens and RBAC enforcement across all APIs.
- **NFR-005 Security – Data Protection**
  All communications shall use TLS 1.2+, and PII shall be stored encrypted at rest in MongoDB.
- **NFR-006 Privacy and Compliance**
  The system shall comply with FERPA for student information and PCI-DSS guidelines for payment processing; no cardholder data shall be stored on servers.
- **NFR-007 Reliability and Recovery**
  The Daily database backups shall ensure a Recovery Point Objective RPO ≤ 24 hours and a Recovery Time Objective (RTO) ≤ 8 hours.
- **NFR-008 Maintainability**
  The codebase shall follow modular design principles and achieve at least 90 % unit test coverage on critical backend modules.
- **NFR-009 Accessibility**
  The web application shall conform to WCAG 2.1 AA accessibility standards to support users with disabilities.
- **NFR-010 Usability**
  The interface shall maintain consistency, use university branding, and require no more than three clicks to access major features *User Engagement Processes*.

Copyright © 2025 Tony D Barber. All rights reserved.

- **NFR-011 Compatibility**
  The system shall be compatible with modern *Supported Browsers* and responsive across *Compatible Devices*.
- **NFR-012 Logging and Observability**
  The backend shall log all major events with *Tracking Information*, expose structured logs, and provide monitoring metrics for API latency and error rates.
- **NFR-013 Containerization and Deployment**
  All system components *System Components* shall be containerized via Docker Compose for consistent deployment on Akamai Cloud.
- **NFR-014 Documentation and API Discoverability**
  The backend shall automatically generate an OpenAPI 3.0 (Swagger) specification for all REST APIs, accessible at /docs.

Table : Quality Attribute Scenario (SEI Template – Performance) (Extra Credit Attempted)

Attribute	Performance Efficiency
Source	Student
Stimulus	Student submits event registration during high load (≥1000 concurrent users)
Artifact	Registration API / Backend Service
Environment	High concurrency (peak event enrollment hours)
Response	The system processes registration within acceptable response time (≤2 seconds)
Response Measure	95% of registration requests meet latency target; no registration failures under 1000 concurrent sessions

Table 6 : Performance Quality Attribute Scenario

Non-Functional Requirements Prioritization : Analytical Hierarchy Process (AHP) (Extra Credit Attempted)

- **Analytical Hierarchy Process -** The Analytic Hierarchy Process (AHP) is a structured decision-making method that uses pairwise comparisons to determine the relative importance (weights) of multiple criteria or attributes, helping prioritize them based on their impact or value.
- **Interpretation**
  - Demonstrates stakeholder-driven, quantitative prioritization of architectural qualities.
- **Why does it fit here ?**
  - AHP lets you quantitatively rank quality attributes by pairwise comparison (0–9 scale).

Copyright © 2025 Tony D Barber. All rights reserved.

- This is great if you want to show numeric weighting and rigor.
- Using the Analytic Hierarchy Process (AHP), stakeholders assigned relative importance values to key quality attributes.

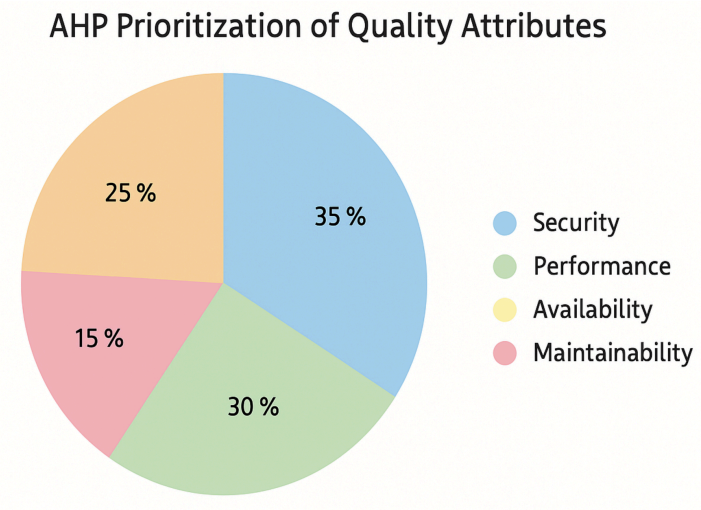


Figure 9 : AHP Prioritization for Non-Functional Requirements

- **Results**
  - The results show **Security (35%)**, **Performance (30%)**, **Availability (20%)**, and **Usability (15%)** as the top priorities influencing architecture decisions.

3.3. Abuse Cases (Security and Misuse Scenarios)

- **AC-01**
  - **Title**: Unauthorized Role Escalation
  - **Source**: Student or Staff user
  - **Stimulus**: Attempts to modify their own privileges to become an Administrator or Event Organizer.
  - **Artifact**: User roles table / authentication module.
  - **Environment**: Normal operation.
  - **System Response**: Deny the request, enforce RBAC validation, log the attempt, and alert the Administrator.
  - **Response Measure**: 100% of unauthorized role escalation attempts blocked; integrity of role assignments preserved.

Copyright © 2025 Tony D Barber. All rights reserved.

Abuse Case Diagram for Unauthorized Role Escalation (Extra Credit Attempted)

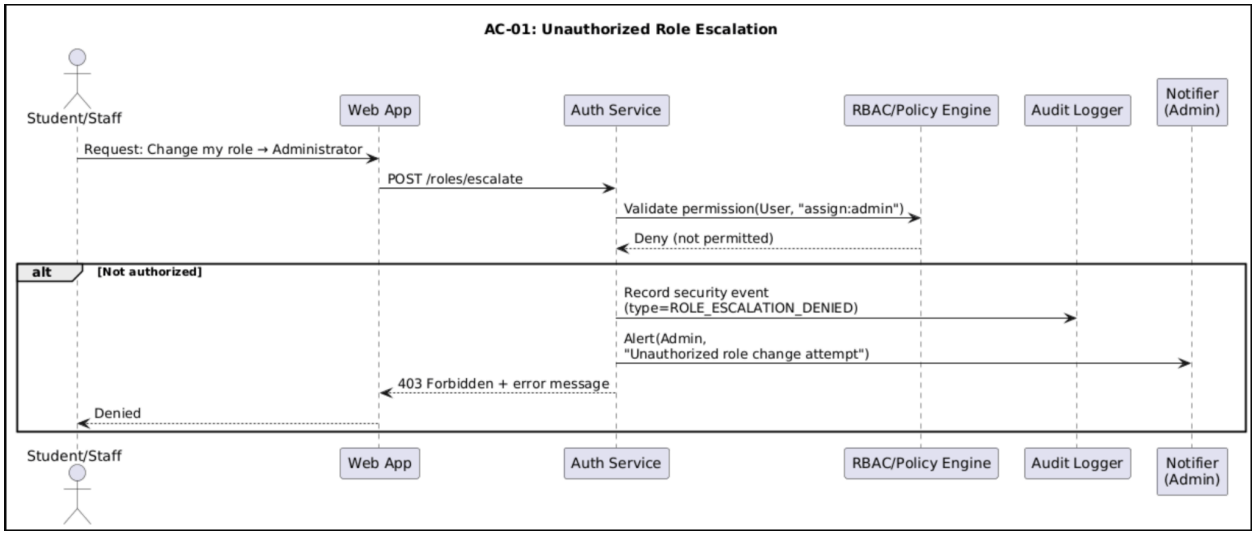


Figure 10 : Abuse Case - 01

- **AC-02**
  - **Title**: Event Data Manipulation by Unauthorized User
  - **Source**: Student or Staff user
  - **Stimulus**: Attempts to edit or delete events not owned by them.
  - **Artifact**: Event management module / event database.
  - **Environment**: Normal operation.
  - **System Response**: Reject modification requests without proper ownership or privilege, log event details, and alert the Event Organizer.
  - **Response Measure**: 100% of unauthorized edit/delete actions denied; no integrity violations in event data.

Copyright © 2025 Tony D Barber. All rights reserved.

Abuse Case Diagram for Event Data Manipulation by Unauthorized User (Extra Credit Attempted)

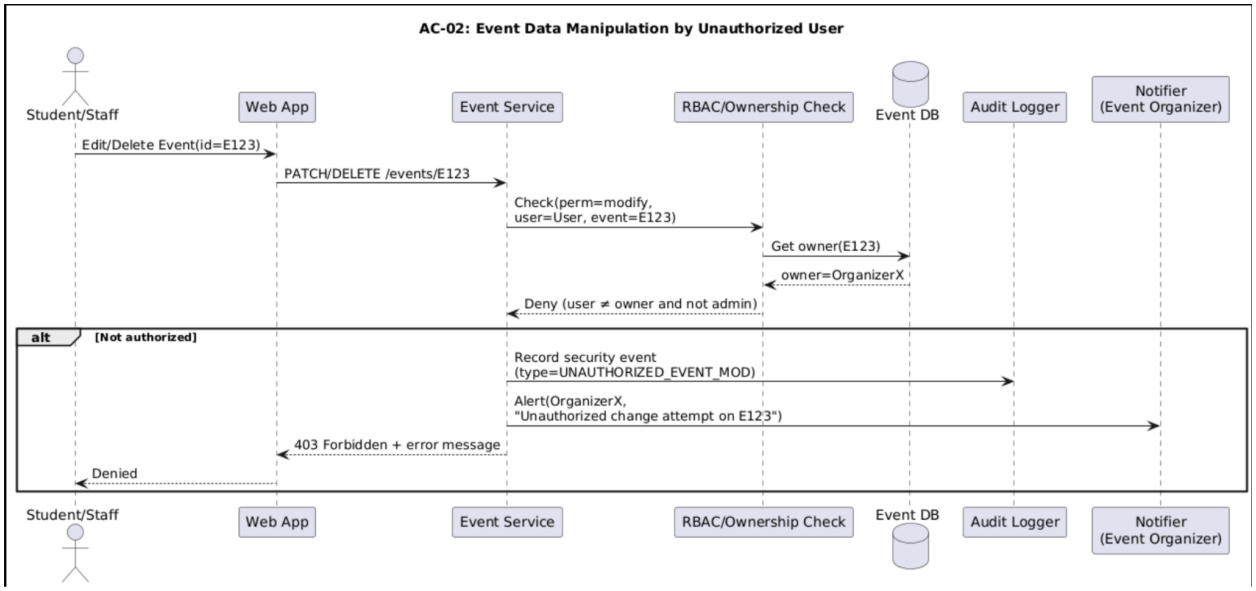


Figure 11 : Abuse Case - 02

- **AC-03**
  - **Title**: Registration Flood (Bot Attack)
  - **Source**: External attacker or automated bot
  - **Stimulus**: Sends large volumes of fake registration requests to fill event capacity.
  - **Artifact**: Registration API / database.
  - **Environment**: Normal or high-traffic operation.
  - **System Response**: Rate-limit requests per IP, enforce CAPTCHA, validate CAS authentication, and log suspicious activity.
  - **Response Measure**: ≥95% of fake registration attempts blocked; availability maintained for legitimate users.
- **AC-04**
  - **Title**: Unauthorized Refund Request
  - **Source**: Student user
  - **Stimulus**: Attempts to request a payment refund for an event they are not registered for.
  - **Artifact**: Payment and refund subsystem.
  - **Environment**: Normal operation.
  - **System Response**: Verify registration ownership, reject invalid refund request, and log transaction attempt.
  - **Response Measure**: 100% of unauthorized refund requests denied; financial integrity preserved.
- **AC-05**
  - **Title**: Fake or Inappropriate Event Creation
  - **Source**: Malicious Event Organizer or compromised Staff account
  - **Stimulus**: Attempts to create fraudulent or inappropriate events.

Copyright © 2025 Tony D Barber. All rights reserved.

    - **Artifact**: Event creation and approval module.
    - **Environment**: Normal operation.
    - **System Response**: Require Administrator approval before publishing; flag suspicious content for review.
    - **Response Measure**: 100% of unapproved or flagged events remain unpublished; system integrity preserved.
- **AC-06**
    - **Title**: Data Exfiltration via Report Exports
    - **Source**: Malicious Staff or Organizer
    - **Stimulus**: Attempts to export large sets of participant data for unauthorized use.
    - **Artifact**: Reporting and analytics module.
    - **Environment**: Normal operation.
    - **System Response**: Limit export access by role, require justification for large data exports, watermark reports, and log all exports.
    - **Response Measure**: 100% of exports logged with user attribution; no unapproved data leakages.
- **AC-07**
    - **Title**: Payment Gateway Forgery
    - **Source**: External attacker
    - **Stimulus**: Sends forged "payment successful" webhook to gain event access without actual payment.
    - **Artifact**: Payment webhook endpoint / transaction records.
    - **Environment**: Normal operation.
    - **System Response**: Verify digital signatures and timestamps of all webhooks; reject unverifiable payloads; alert Administrator.
    - **Response Measure**: 100% of forged webhook events rejected; payment integrity preserved.
- **AC-08**
    - **Title**: Feedback Tampering
    - **Source**: Malicious Organizer or Student
    - **Stimulus**: Attempts to alter or delete feedback submissions.
    - **Artifact**: Feedback collection and database module.
    - **Environment**: Normal operation.
    - **System Response**: Restrict edit/delete access, maintain immutable feedback records, and log all modification attempts.
    - **Response Measure**: 100% of unauthorized feedback changes prevented; data integrity maintained.
- **AC-09**
    - **Title**: Calendar Feed Injection
    - **Source**: External attacker
    - **Stimulus**: Attempts to inject malicious scripts or URLs into public calendar feeds.
    - **Artifact**: Calendar feed generator / event descriptions.
    - **Environment**: Normal operation.

Copyright © 2025 Tony D Barber. All rights reserved.

    - **System Response**: Sanitize and escape all content before feed generation; block unsafe HTML/script content.
    - **Response Measure**: 100% of calendar feed entries sanitized; no malicious code injection possible.
- **AC-10**
    - **Title**: Session Hijacking
    - **Source**: External attacker
    - **Stimulus**: Attempts to reuse or steal session cookies to impersonate another user.
    - **Artifact**: Authentication and session management module.
    - **Environment**: Normal operation or shared device use.
    - **System Response**: Enforce secure session cookies (HTTPOnly, SameSite, short TTL), rotate tokens on privilege change, and log anomalies.
    - **Response Measure**: 100% of reused or stolen session tokens invalidated; confidentiality and integrity maintained.

Summary of Threat Coverage (Extra Credit Attempted)

Category	Threat Focus	Abuse Case Covered
Confidentiality	Unauthorized access or data leakage	AC-01, AC-06, AC-10
Integrity	Unauthorized modification or forgery	AC-02, AC-04, AC-07, AC-08
Availability	Denial of service or registration flood	AC-03, AC-09
Accountability	Logging, monitoring, traceability	All cases (logging + alerting)

Table 7 : Summary of Threat Coverage

3.4. Requirements Traceability Matrix

- Each requirement in this document is assigned a unique identifier (e.g., FR-#, NFR-#, AC-#) for precise reference and tracking.
- The Requirements Traceability Matrix (RTM) establishes and maintains bidirectional traceability between the system requirements, their sources, the design components that implement them, and the test cases that verify them.
- The RTM ensures that:
    - Every requirement can be traced forward to one or more corresponding design modules and test cases (forward traceability).
    - Every test case can be traced backward to its originating requirement (backward traceability).
    - The matrix is maintained as a living document, continuously updated as requirements evolve across project iterations.
- The RTM includes the following key columns:

Copyright © 2025 Tony D Barber. All rights reserved.

- Requirement ID → Unique reference number for each requirement (e.g., FR-001, NFR-005).
- Requirement Description → Concise statement summarizing the requirement.
- Source → Originating stakeholder, business driver, or regulation.
- Design Reference → Module, feature, or architecture element addressing the requirement.
- Test Case(s) → Identifier(s) for the test cases verifying the requirement.
- Status → Progress stage of the requirement (Proposed, Approved, Implemented, Verified).

Requirements Traceability Matrix Table (Extra Credit Attempted)

Requirements Traceability Matrix

Copyright © 2025 Tony D Barber. All rights reserved.

4. External Interfaces Requirements

Validation Phase (Extra Credit Attempted)

- Checking that our specification matches the user's requirements

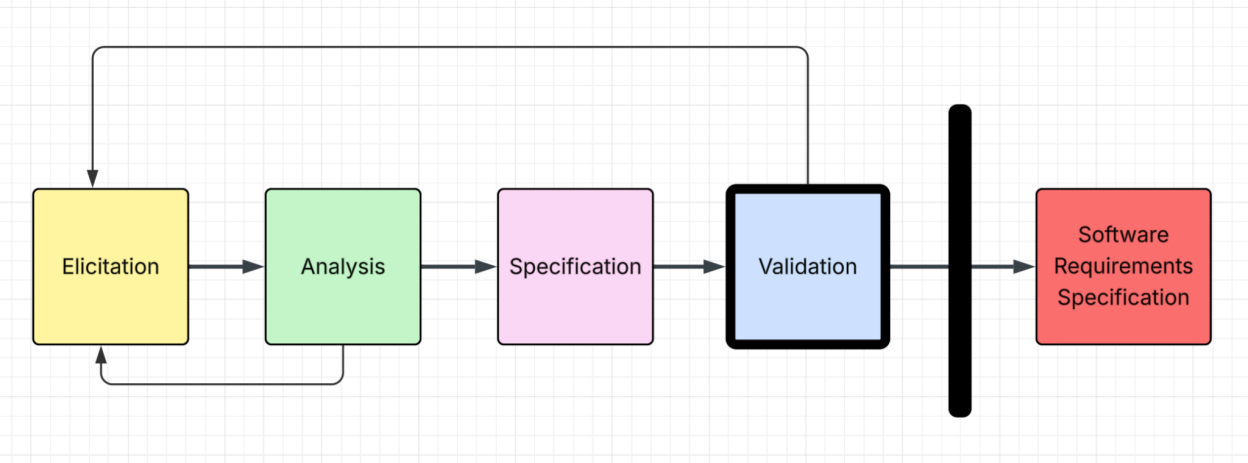


Figure 12 : Validation Phase

4.1. User Interfaces

Landing Page

The **Terrapin Events** **landing page** serves as the primary public entry point for the system, designed to attract, inform, and engage users before authentication. It provides a clean, modern interface that emphasizes upcoming and featured campus events with visual appeal. The layout features a ***Hero Banner*** highlighting key or trending events, and a Login-in button for authentication.

Clear **call-to-action** ***CTA*** elements, such as "Log in with CAS," and "View Details" are prominently placed to encourage immediate engagement.

Copyright © 2025 Tony D Barber. All rights reserved.



Figure 13 : Landing Page

Authorized Users Dashboard

Upon logging in, users are directed to the **Home Page**, which serves as the central hub for exploring and managing events. Navigation is handled through a ***Hamburger Menu*** located in the upper corner, allowing quick access to key sections such as ***Hamburger Menu Options***.

The **Home Page** lists all upcoming events organized by date. As the user scrolls, additional future events automatically load into view, ensuring continuous discovery without page reloads. A **search bar** is prominently placed at the top for locating specific events, accompanied by a **filter control** that allows sorting and refinement by category, date, venue, or event type (e.g., free or paid).

Each event appears as an **interactive card** displaying key details such as title, organizer, date/time, venue, capacity, remaining vacancies, and any active waitlist. If the user is already registered for an event, a distinct **visual indicator** (for example, a badge or color highlight) marks it as registered. Clicking on an event card navigates to the **Event Detail Page**, where expanded information is displayed including a detailed description, event tags, map links, accessibility provisions, refund or transfer policies, and contact information.

The ***Authorised Users*** **Dashboard** further enhances usability by featuring “Upcoming Events,” “Recommended/Trending” sections, and **quick filters** for fast navigation. Users can also perform ***One Click Actions***.

Copyright © 2025 Tony D Barber. All rights reserved.

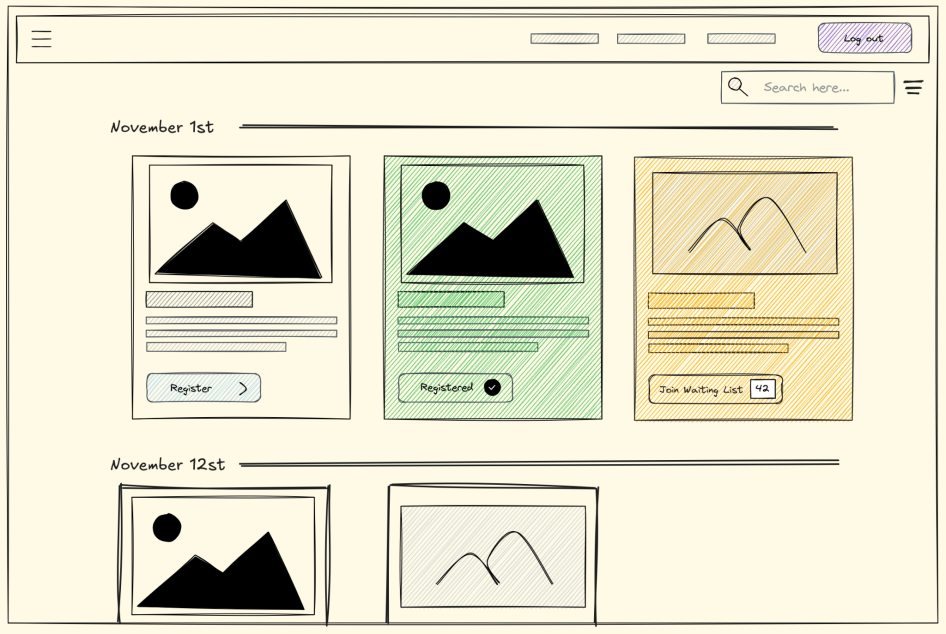


Figure 14: Authorized User - Home

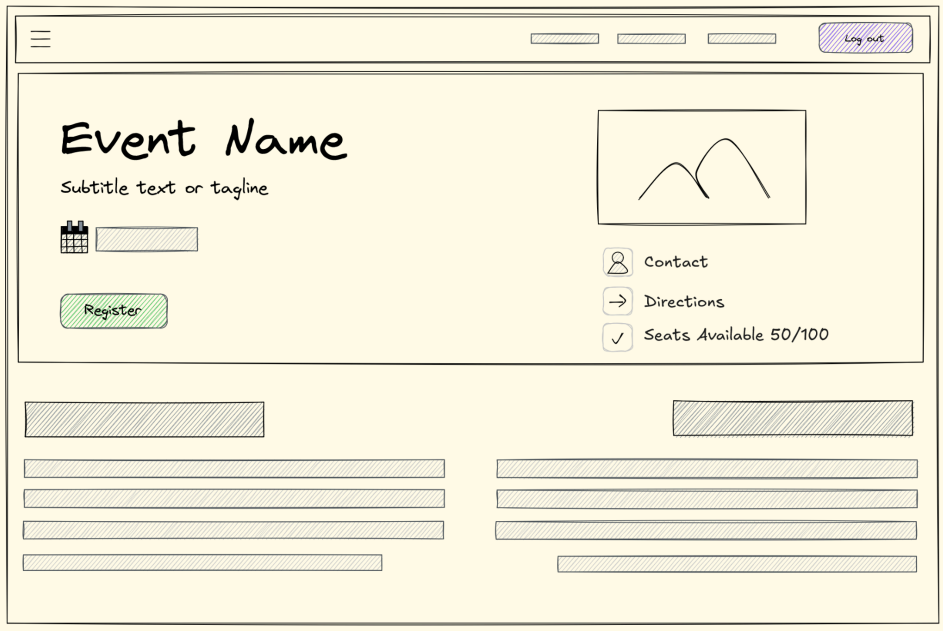


Figure 16: Authorized User - Event Detail

Organizer Console

The **Organizer Console** provides event organizers with tools to create, manage, and monitor their events through an intuitive and structured interface. Upon logging in, organizers are directed to their **Home Page**, which displays all upcoming events they have created. These events are presented as **interactive cards**, similar in layout to the student/faculty dashboard, showing key

Copyright © 2025 Tony D Barber. All rights reserved.

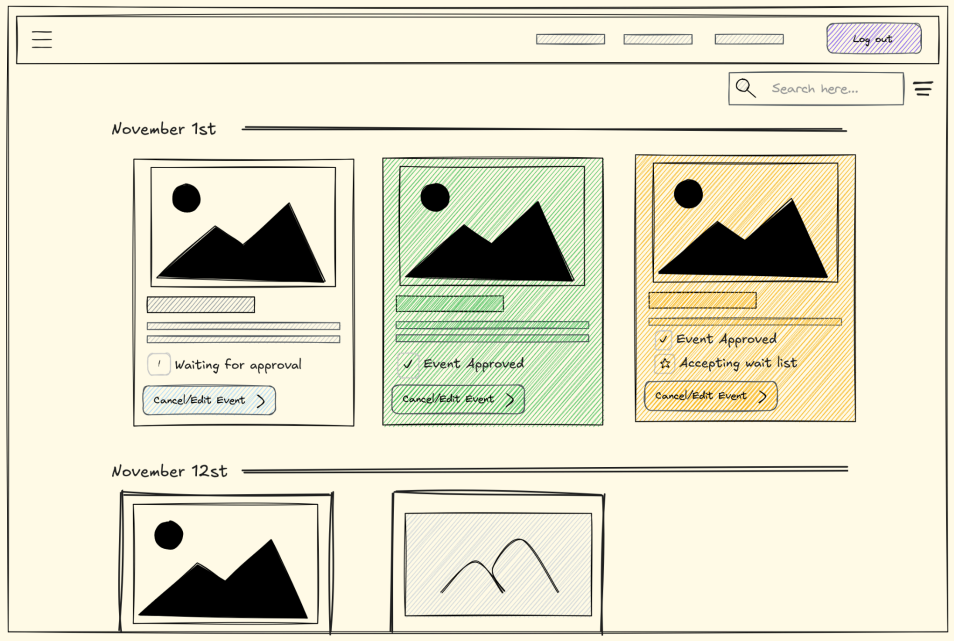
details such as event title, date/time, venue, capacity, remaining vacancies, and active waitlists. Each card also includes an additional **“Cancel Event”** option, allowing organizers to withdraw an event directly from the console.

A **search bar** is provided at the top of the page for quick lookup of specific events. Navigation is handled via a **hamburger menu**, giving organizers direct access to sections such as *Organized Events*, *Create Event*, *Calendar*, and *Log Out*.

When creating or editing events, organizers follow a guided **multi-step wizard** that simplifies the event setup process through sequential stages:

- **Basics** → Event title, description, department, and category
- **Schedule/Venue** → Date, time, and location details
- **Capacity/Waitlist** → Maximum attendees and waitlist settings
- **Ticketing/Price** → Fee structure and payment options
- **Media** → Upload of event banners or promotional images
- **Policies** → Event-specific rules and refund policies
- **Preview** → Final review before publishing

All event cards navigate the user to the **Event Detail Page** allowing organizers to **export data in CSV format** for reporting or administrative use.



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 17 : Organizer - Home

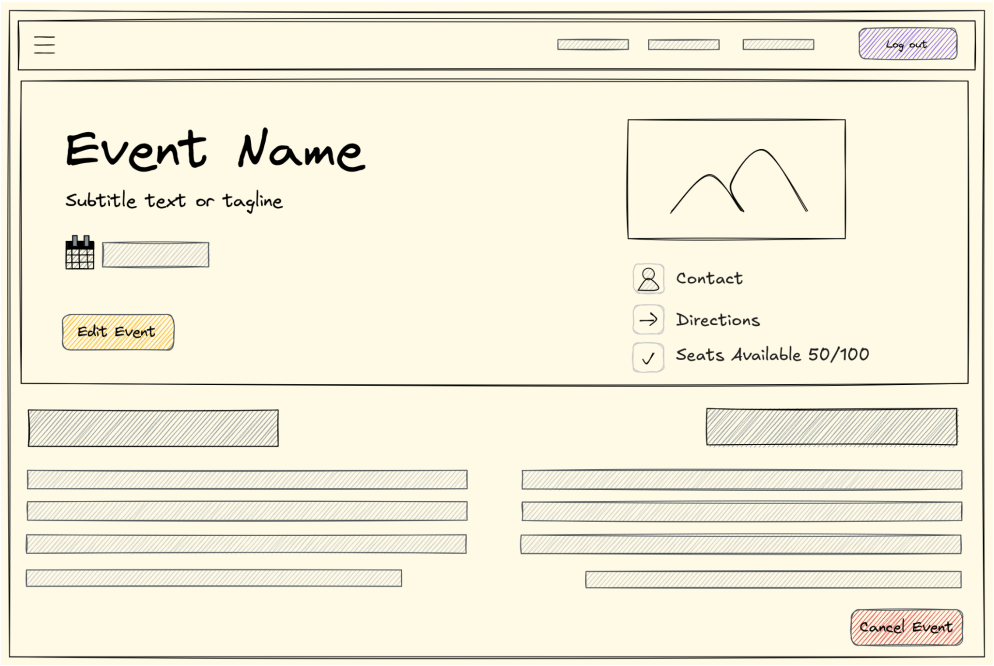


Figure 18 : Organizer - Event Detail

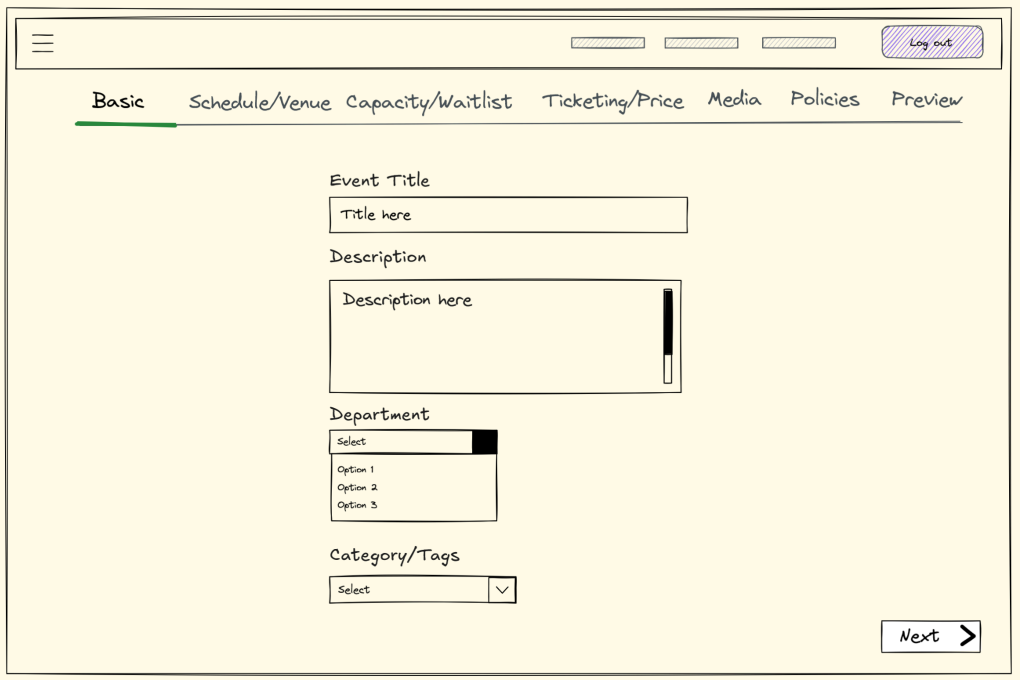


Figure 19 : Organizer - Multi-step Wizard

Admin Console

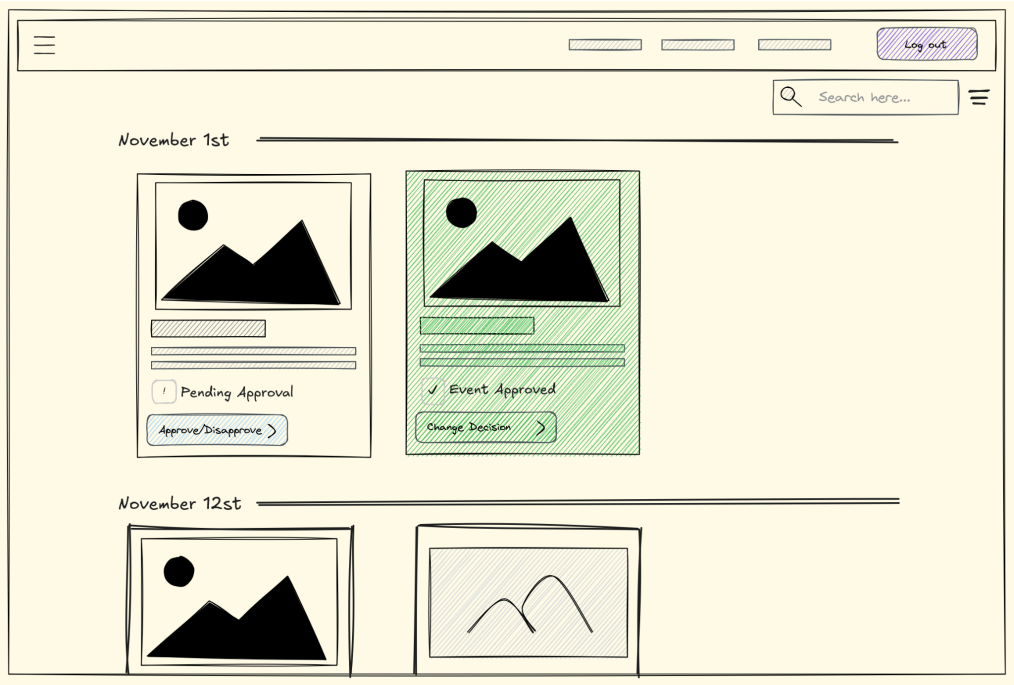
Copyright © 2025 Tony D Barber. All rights reserved.

The **Admin Console** provides system administrators with comprehensive control over event approvals, organizer role management, and policy compliance within the **Terrapin Events** platform. Upon logging in, administrators are directed to their **Home Page**, which displays all **pending event approvals**. Each event is presented as an **interactive card**, consistent with the layout of other dashboards, showing essential details such as event title, organizer name, date/time, and submission status. Clicking a card opens the **Event Detail Page**, where administrators can review all event information before making a decision.

Within the **Event Detail** view, administrators have the option to **Approve** or **Disapprove** an event. If an event is disapproved, the system prompts the administrator to provide a **reason or feedback comment**, ensuring transparency and clear communication. These comments are automatically **emailed to the event organizer** to inform them of the decision and necessary revisions.

In addition to event approval management, the Admin Console includes an **Assign Organizers** module. This page enables administrators to **search for users** (students or faculty) by their ***UID*** and **grant or revoke organizer privileges** as needed. This feature ensures proper role governance and accountability across the platform.

The **Admin Console** uses a ***Hamburger Menu*** for navigation, providing access to key sections such as *Pending Approvals*, *Approved Events*, *Current Organizers*, *Assign Organizers*, and *Log Out*.



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 20: Admin - Home

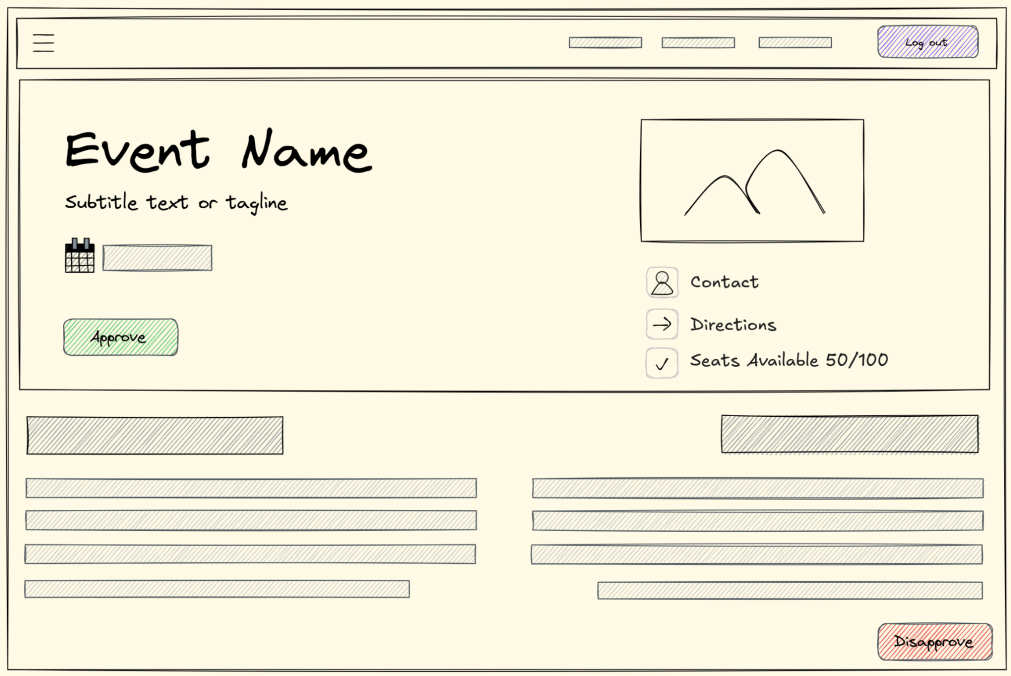


Figure 21: Admin - Event Detail

Branding

The design follows the university's branding guidelines, using consistent colors, typography, and imagery to reinforce institutional identity.

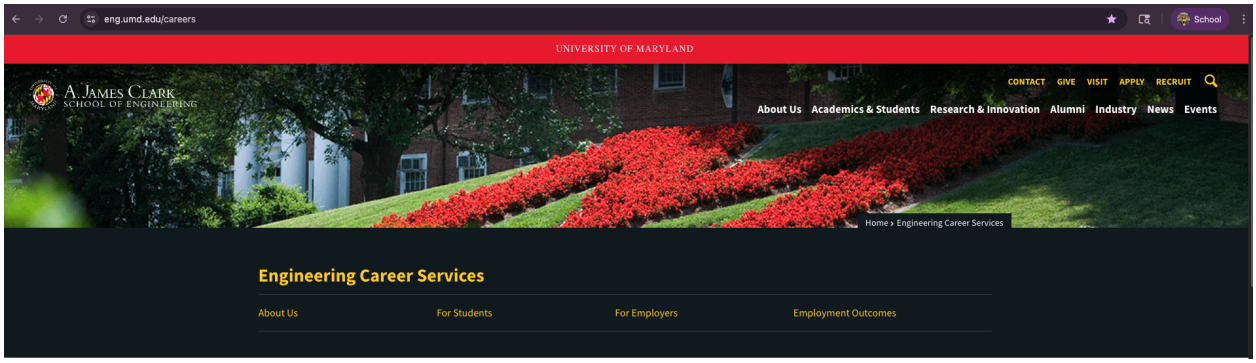


Figure 22: Branding Sample

Accessibility (WCAG/ADA)

- Conform to **WCAG 2.1 AA** and campus ***ADA*** expectations, semantic HTML, proper landmarks, focus order, visible focus states, skip-to-content, keyboard-operable menus, form errors linked to inputs, alt text for images.
- Color contrast ≥ 4.5:1 for text, no information conveyed by color alone.
- Live regions for async updates (***ARIA***)

Copyright © 2025 Tony D Barber. All rights reserved.

- Screen-reader friendly calendar/list toggles

4.2. Software Interfaces

UMD CAS

- **Purpose:** Authenticate all users (students, faculty, organizers) with role-based access.
- **Constraints:** Must use university **CAS;** campus IT provides client registration and metadata. Must follow university security policies, no storage of campus credentials locally.

Payment Gateway (university-approved)

- **Purpose:** Process payments for ticketed/premium events.
- **Constraints:** Must follow **r**edirect flow, tokenize card data, ***PCI DSS*** compliance, system **must not** store ***PAN/CVV***.

Email/Notification Service

- **Purpose:** Communication regarding confirmations, reminders, updates, and cancellations.
- **Constraints:** campus SMTP or approved provider, acceptable rate limiting & retries.

Calendar Integration

- **Purpose:** personal calendar, add-to-calendar and campus-wide publishing.
- **Notes:** The calendar module must support **ICS/iCal** files.

Analytics/Reporting Export

- **Purpose:** CSV/JSON exports for attendance/engagement, FERPA-aware aggregation and data retention.

Browser Targets

- Chrome, Firefox, Edge, Safari (all campus-supported versions).

4.3. Hardware Interfaces

- The Terrapin Events system is a fully cloud-based web application and does not require any dedicated hardware interfaces. All interactions occur through standard computing devices (e.g., laptops, tablets, mobile phones) via web browsers.

4.4. Communication Interfaces

Data Exchange Patterns

- **Primary APIs:** RESTful endpoints with ***JSON*** payloads.
- **Webhooks (optional/future):** outbound event notifications (status changes, capacity updates) to attendees and organizers.
- **Batch exports:** CSV/JSON downloads for reports; ICS feeds for calendars.

Copyright © 2025 Tony D Barber. All rights reserved.

Protocols, Transport, and Security

- **Transport: HTTPS (TLS 1.2+)** only, HSTS enabled, secure cookies (HttpOnly/SameSite), CSRF protection on state-changing requests.
- **Authentication/Authorization**: UMD ***CAS*** for login, application roles & least-privilege policy are implemented server-side
- **Compliance & Privacy:** ***FERPA*** for student/faculty data, ***PCI DSS*** for payments, audit logging for privileged actions, PII minimization and retention limits consistent with campus policy.
- **Reliability:** Idempotent PUT/PATCH/ where feasible, exponential backoff on outbound calls
- **Performance:** Typical request ≤ 800ms under normal load.

Copyright © 2025 Tony D Barber. All rights reserved.

5. Use Cases Diagrams and Narrative / Scenarios

UC-01: Sign in with CAS SSO

Primary Actor: ***Authorised users***

Secondary Actor: UMD ***CAS***
Goal: Establish an authenticated application session.
Preconditions: User has a valid UMD Directory ID.
Trigger: User selects **"Log in with CAS"**.

Main Flow:

1. The user selects **Login with CAS**.
2. The system redirects the user to **UMD CAS**.
3. The user enters credentials and submits.
4. CAS authenticates the user and returns an assertion.
5. The system validates the assertion and creates an application session.
6. The system loads the ***role-based dashboard***.

Alternate Flows:

5a. Assertion invalid/expired:

- The system prompts re-authentication.
- *Return to step 2.*

Exception Flows:

1a. CAS page fails to load:

- The system displays a retry message.
- Return to step 2.

2a. CAS timeout/unavailable:

- The system notifies the user and offers Retry.
- *Return to step 2.*

3a. Invalid credentials:

- The system shows error and allows up to three retries.
- *Return to step 3.*

Postconditions: Authenticated session is active; user dashboard is displayed.
Traceability: FR-001, FR-002, FR-003.

Copyright © 2025 Tony D Barber. All rights reserved.

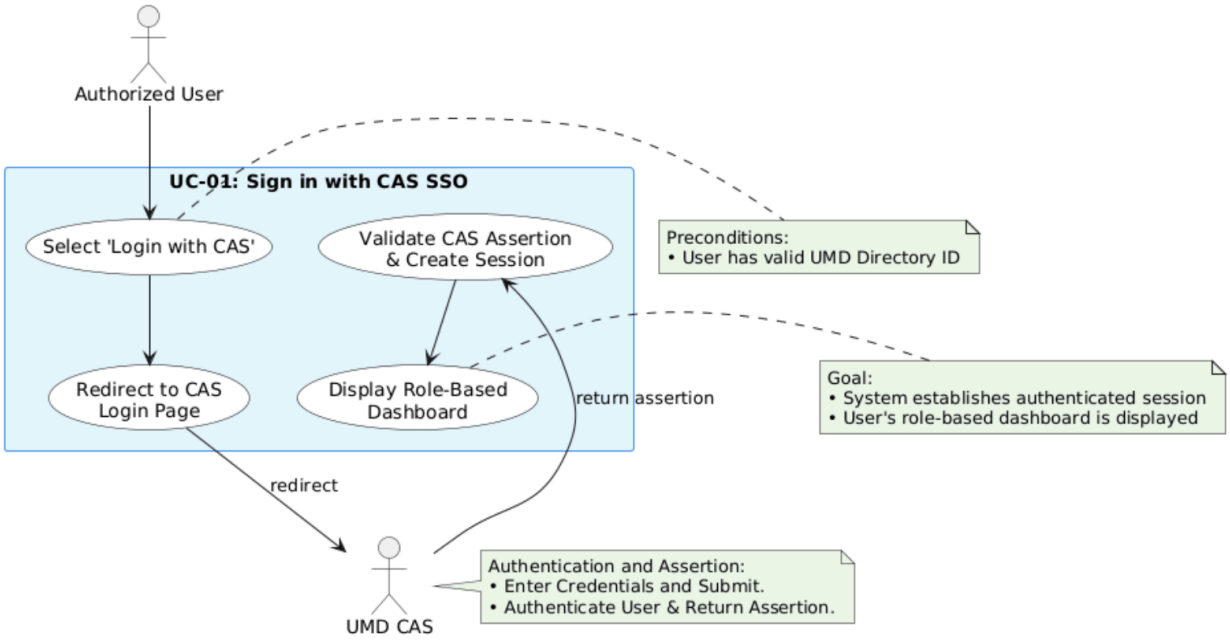


Figure 23: Use Case 001 - Sign with CAS SSO

UC-02 Create and Submit Event for Approval

Primary Actor: Event Organizer
Goal: Submit a new event for administrator review.
Preconditions: Organizer is authenticated and authorized.
Trigger: Organizer selects “Create Event”.

Main Flow:

1. Organizer selects **Create Event**.
2. The system opens the **Event Creation Wizard** with a blank form.
3. The organizer enters required ***Event Details***.
4. The system validates inputs and **saves a draft**.
5. Organizer selects **Submit for Approval**.
6. System sets status to **Pending Approval** and **notifies Admin**.

Alternate Flows:

1a. Organizer reopens a pending event to edit:

- The system creates a **new version**.
- *Return to step 3.*

6a. Notification service unavailable:

- System queues notifications ; event remains **Pending Approval**.

Copyright © 2025 Tony D Barber. All rights reserved.

- *No user action; flow terminates successfully.*

Exception Flows:

3a. Validation fails (missing/invalid fields):

- The system highlights errors.
- *Remain at step 3.*

4a. Network failure while saving:

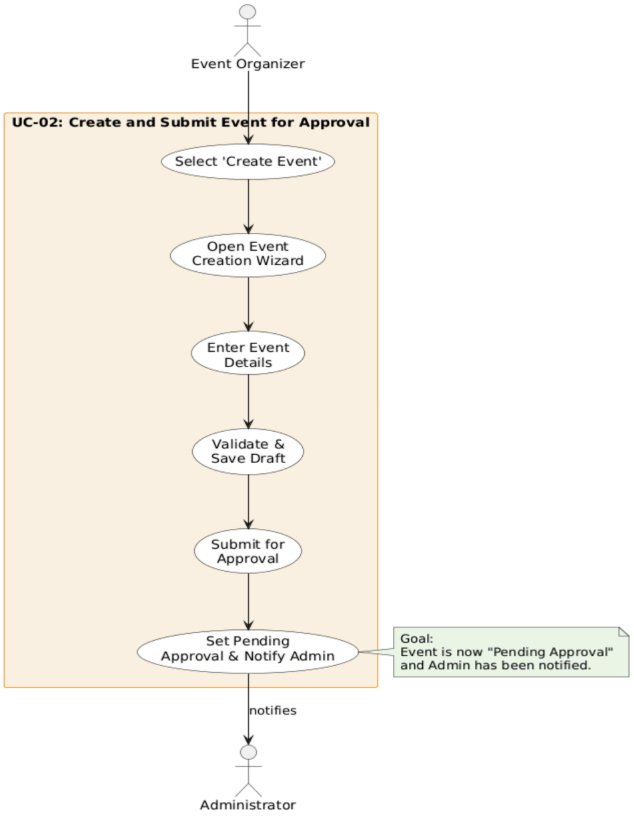
- System stores draft locally on the client side and schedules retry.
- *When restored, continue at step 5.*

5a. Required compliance fields (e.g., policy/visibility) missing:

- System blocks submission and lists missing fields.
- *Return to step 3.*

Postconditions: Event status = Pending Approval; Admin notified via email.

Traceability: FR-005, FR-006, FR-007, FR-008.



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 24: Use Case - 002- Create and Submit Event for Approval

UC-03 Approve Event

Primary Actor: Administrator
Goal: Validate and publish events.
Preconditions: Admin is logged in to the system and Event is Pending for Approval.
Trigger: Admin opens event review page.

Main Flow:

1. Admin opens the event that needs **Approval**.
2. System loads event details and version history.
3. Admin chooses **Approve**.
4. System **publishes** the event.
5. The system **notifies** the organizer and subscribers.

Alternate Flows:

3a. Admin selects **Reject** with feedback:

- The system sets status to **Changes Requested** and sends feedback to the Organizer.
- *Flow terminates (Organizer may edit and resubmit).*

Exception Flows:

1a. Event already actioned (approved/rejected by someone else):

- The system shows read-only state and the prior decision.
- *Flow terminates.*

2a. Policy/compliance violation detected:

- The system blocks publication and requires correction.
- *Flow terminates (Organizer must resubmit).*

4a. Concurrent approvals/rejections issued:

- The resources are in FIFO order and services are locked for those accessing the resource first.
- As our system implements ACID transactions *impartial data will be avoided.*
- *Return to step 1.*

Postconditions: Event either Published or Returned with feedback.

Traceability: FR-008, FR-009, FR-020.

Copyright © 2025 Tony D Barber. All rights reserved.

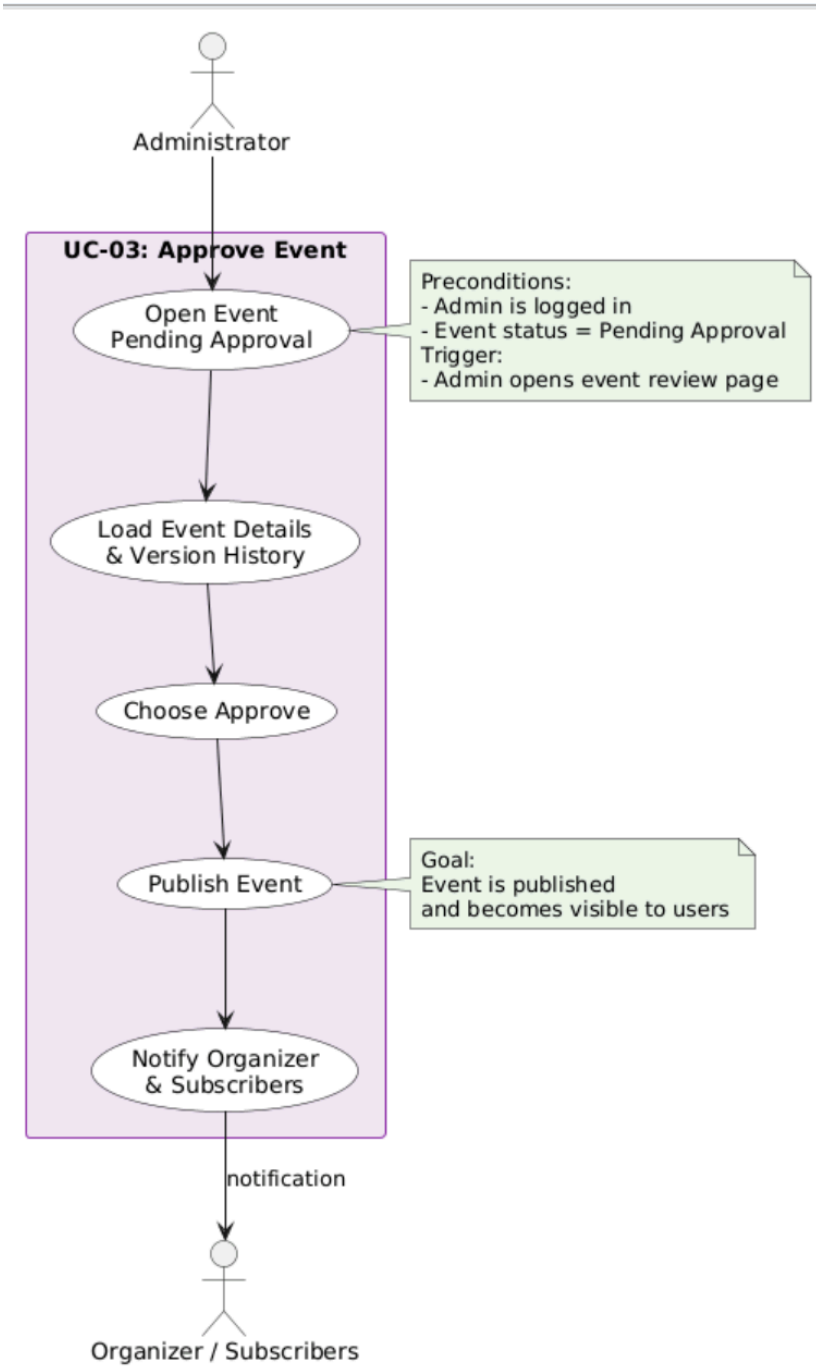


Figure 25: Use Case - 003- Approve Event

UC-04 Discover and Register for Event

Primary Actor: ***Authorised Users***

Secondary Actor: ***Calendar Types***
Goal: Find and register for events.

Copyright © 2025 Tony D Barber. All rights reserved.

Preconditions: User authenticated; events are Published.
Trigger: User selects “Search Events” or “Register”.

Main Flow:

1. User opens **Search Events**.
2. The system displays results (filters/facets).
3. The user selects an event to **view details**.
4. User clicks **Register**.
5. The system checks **capacity and eligibility**.
6. The system creates a **registration** and displays confirmation.
7. The system offers **Add to Calendar** and sends a confirmation email.

Alternate Flows:

5a. Event full:

- The system adds the user to **Waitlist** and confirms position.
- *Flow terminates.*

6a. Paid event requires checkout:

- The user selects **Checkout.**
- System directs to **UC-05 Pay for Ticketed Event.**
- *On successful payment, continue at step 7.*

7a. User later cancels registration:

- The system frees the seat and **promotes the first waitlisted** user (if any).
- *Flow terminates for this user.*

Exception Flows:

1a. No results found:

- The system suggests loosening filters or alternative dates.
- *Flow terminates.*

4a. User already registered:

- System shows **Already Registered** message.
- *Flow terminates.*

5b. Eligibility checks fail (role, age, prerequisites):

- The system blocks registration and provides reasoning.
- *Flow terminates.*

Postconditions: Registration record created or waitlist entry logged.

Traceability: FR-010 - FR-013, FR-016, FR-017.

Copyright © 2025 Tony D Barber. All rights reserved.

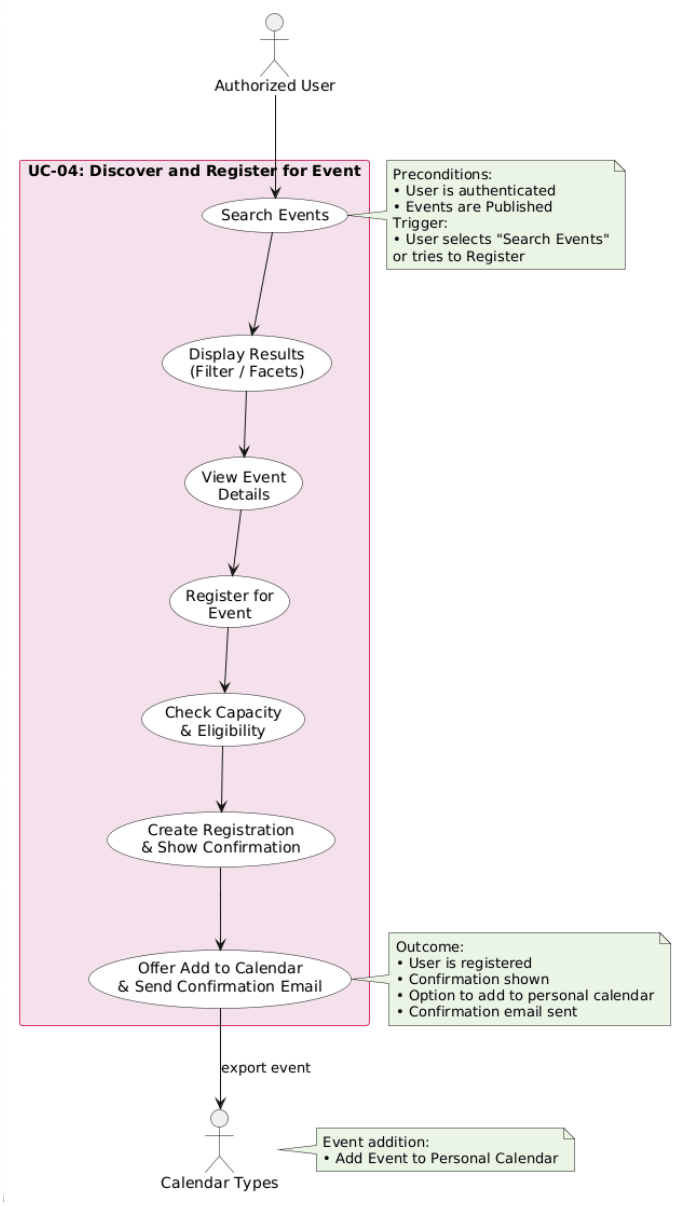


Figure 26: Use Case - 004- Discover and Register for Event

UC-05 Pay for Ticketed Event

Primary Actor: ***Authorised Users***

Secondary Actor: Payment Gateway
Goal: Handle ticket payment.
Preconditions: User registered for paid event; gateway online.
Trigger: User selects “Checkout”.

Main Flow:

Copyright © 2025 Tony D Barber. All rights reserved.

1. The user selects **Checkout** from the registration.
2. System redirects to the **Payment Gateway**.
3. The user enters payment details and submits.
4. Gateway **authorizes** payment and returns responses.
5. The system **records** the transaction and updates registration to **Paid**.
6. The system displays a **receipt** and sends email confirmation.

Alternate Flow:

5a. User requests "Refund" before the deadline:

- System routes to Organizer/Admin for review.
- **5a.i.** Refund **approved**: System posts refund and email confirmation. *Flow terminates.*
- **5a.ii.** Refund **denied**: System notifies user with reason. *Flow terminates.*

Exception Flows:

2a. Gateway timeout/unavailable:

- The system notifies the user and enables **Retry**.
- *Return to step 2.*

4a. Payment declined:

- The system informs the user and allows a different payment method.
- *Return to step 3.*

4b. Suspicious/forged callback detected:

- The system rejects the callback, logs an incident, and keeps registration **Pending Payment**.
- *Return to step 2 for retry.*

Postconditions: Transaction record updated with status (Paid / Refunded).

Traceability: FR-014 - FR-016, FR-020.

Copyright © 2025 Tony D Barber. All rights reserved.

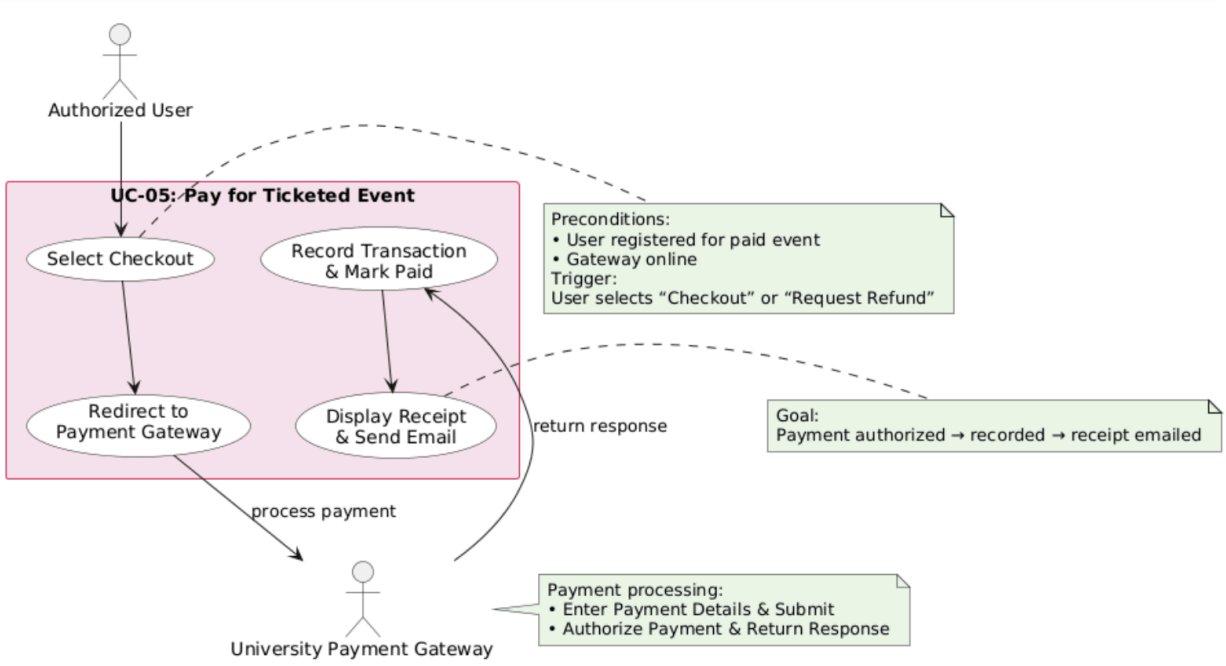


Figure 27: Use Case - 005- Pay for Ticketed Event

Copyright © 2025 Tony D Barber. All rights reserved.

6. Quality Attribute Scenarios

QAC - 001 - Performance

Example: NFR-001 → The system shall respond to all *User-Initiated Operations* within 800 milliseconds for 99.999 % of requests under normal load.

- **P1 - Search latency**
  - **Source:** Student
  - **Stimulus:** Submits keyword search with date/category filters
  - **Artifact:** Event Catalog & Search API
  - **Environment:** Normal load, production data (Akamai Connected Cloud)
  - **Response:** Return first page with counts and facets
  - **Response Measure:** p95 end-to-end ≤ **800 ms**; p99 ≤ **3.0 s**.
- **P2 - Registration confirmation**
  - **Source:** Authenticated attendee
  - **Stimulus:** Clicks "Register" for an event with capacity
  - **Artifact:** Registration service & database
  - **Environment:** Normal load
  - **Response:** Create registration, hold seat, show confirmation, enqueue email
  - **Response Measure:** Confirmation screen ≤ **800 ms**; confirmation email **enqueued ≤ 5 s**.
- **P3 - Checkout (paid events)**
  - **Source:** Attendee paying by card
  - **Stimulus:** Submits payment at checkout
  - **Artifact:** Payments adapter & order workflow
  - **Environment:** Normal load
  - **Response:** Tokenize card, authorize payment, finalize order
  - **Response Measure:** p95 checkout completion ≤ **4.0 s**; authorization success ≥ **99.5%** when gateway available.

QAC - 002 - Availability & Reliability

Example: NFR-003 → The system shall maintain ≥ 99.99 % uptime per month, excluding scheduled maintenance windows.

- **A1 - Uptime *SLO***
  - **Source:** Any user during term
  - **Stimulus:** Requests any core page
  - **Artifact:** Production stack
  - **Environment:** Steady state; planned maintenance excluded
  - **Response:** Service remains reachable and functional
  - **Response Measure:** Monthly availability ≥ **99.0%.**
- **A2 - External gateway outage**
  - **Source:** Payment provider
  - **Stimulus:** Returns 5xx/timeout during checkout
  - **Artifact:** Payments adapter
  - **Environment:** Peak hour
  - **Response:** Fail gracefully, preserve cart, prevent duplicate charges, log incident

Copyright © 2025 Tony D Barber. All rights reserved.

    - **Response Measure:** Clear message ≤ **2 s**; **0** duplicate charges; ≤ **3** retries within **60 s**; order state consistent.
- **A3 - Backup & restore**

Example: NFR-007 → The Daily database backups shall ensure a Recovery Point Objective RPO ≤ 24 hours and a Recovery Time Objective (RTO) ≤ 8 hours.

    - **Source:** Ops
    - **Stimulus:** Requests restore to last good point
    - **Artifact:** DB & object storage
    - **Environment:** DR drill or incident
    - **Response:** Restore data/services; verify integrity
    - **Response Measure : RPO ≤ 24 h**; **RTO ≤ 8 h** (updated from 4 h per NFR-007); integrity check reports **0** failed constraints.

QAC - 003 - Security & Compliance

- **S1 - *RBAC* enforcement(*CAS/JWT*)**

Example: NFR-004 → All user sessions shall be authenticated through CAS SSO and protected via JWT tokens and RBAC enforcement across all APIs.

    - **Source:** Organizer with limited scope
    - **Stimulus:** Attempts to edit another organization's event
    - **Artifact:** Admin/Event APIs, policy engine (CAS SSO + JWT session)
    - **Environment:** Normal operation
    - **Response:** Deny, show reason, write immutable audit log (user, role, IP, payload)
    - **Response Measure:** ≥ **99%** out-of-scope edits blocked; audit entry written **≤ 1 s**; **0** unauthorized rows updated.
- **S2 - Sensitive export guard(PII)**

Example: NFR-005 → All communications shall use TLS 1.2+, and PII shall be stored encrypted at rest in MongoDB.

    - **Source:** Organizer
    - **Stimulus:** Requests export containing student PII
    - **Artifact:** Reports/Export module
    - **Environment:** Normal
    - **Response:** Require elevated role + approval; mask PII by default; log purpose
    - **Response Measure: 0** PII exports without proper role & approval; **100%** exports logged (requester, purpose, timestamp).
- **S3 - Card data handling(PCI-DSS)**

Example: NFR-006 →The system shall comply with FERPA for student information and PCI-DSS guidelines for payment processing; no cardholder data shall be stored on servers.

    - **Source:** Attendee entering card
    - **Stimulus:** Submits card details at checkout

Copyright © 2025 Tony D Barber. All rights reserved.

- **Artifact:** Checkout UI & gateway fields
- **Environment:** Normal
- **Response:** Hosted fields/tokenization; **no PAN/CVV stored**; scrub logs; transport via **TLS 1.2+ (prefer 1.3)**
- **Response Measure: 0** cardholder data stored or logged; quarterly review finds **0** critical PCI issues.

QAC - 004 - Usability & Accessibility

- **U1 - "Find & register" task**

Example: NFR-009 → The web application shall conform to **WCAG** 2.1 AA accessibility standards to support users with disabilities.

- **Source:** First-time student
- **Stimulus:** Finds an event and completes registration
- **Artifact:** Catalog, event page, registration flow
- **Environment:** Desktop & mobile browsers
- **Response:** Complete task without assistance
- **Response Measure :** ≥ **90%** of test users finish in ≤ **3 clicks** from catalog (≤ **5** with login) and ≤ **90 s**; **SUS ≥ 80**.

- **U2 - Accessibility conformance**

Example: NFR-010 → The interface shall maintain consistency, use university branding, and require no more than three clicks to access major features of *User Engagement Processes*.

- **Source:** Screen-reader/keyboard-only user
- **Stimulus:** Navigates catalog → event → register
- **Artifact:** Front-end UI components
- **Environment:** NVDA/JAWS + Chrome/Firefox
- **Response:** Fully operable and understandable
- **Response Measure: WCAG 2.1 AA**; automated scan finds **0** critical issues; manual audit passes key journeys.

- **U3 - Compatibility**

Example: NFR-011 → The system shall be compatible with modern *Supported Browsers* and responsive across *Compatible Devices*.

- **Source:** Student or staff
- **Stimulus:** Uses latest Chrome, Edge, Firefox, or Safari on mobile/tablet/desktop
- **Artifact:** Web UI
- **Environment:** Modern browsers, responsive layouts
- **Response:** Features render and function consistently
- **Response Measure:** All key journeys pass on supported browsers/devices; **0** critical browser-specific defects in release.

QAC - 005 - Scalability

Copyright © 2025 Tony D Barber. All rights reserved.

- **C1 - Flash registration window(autoscaling)**

Example: NFR-002 → The system shall support *Horizontal Scaling* using containerized deployments on Akamai Connected Cloud to handle 2× baseline concurrent traffic without service degradation.

  - **Source:** Campus-wide event drop
  - **Stimulus:** Traffic surge to **≥ 2× baseline concurrent sessions** (autoscaling enabled on Akamai Connected Cloud)
  - **Artifact:** Web/App tier, DB, cache, queue
  - **Environment:** 15-minute peak
  - **Response:** Maintain SLOs; degrade non-critical features if needed
  - **Response Measure :** At **≥2× baseline**, p95 search ≤ **2.5 s**, p95 registration ≤ **3.0 s**; **5xx < 0.5%**; queue depth stabilizes **≤ 10 min**.
    (If you keep 5,000 as your defined 2× baseline, note it in the test plan.)

QAC - 006 - Maintainability & Deployability

Example: NFR-008 → The codebase shall follow modular design principles and achieve at least 90 % unit test coverage on critical backend modules.

- **M1 - MTTR Sev-2**
  - **Source:** Monitoring or user report
  - **Stimulus:** Sev-2 defect in production
  - **Artifact:** Codebase, CI/CD, observability
  - **Environment:** Business hours
  - **Response:** Diagnose, patch, test, deploy via pipeline
  - **Response Measure :** Median **MTTR ≤ 8 h**; regression test added; postmortem published **≤ 48 h**.
- **M2 - Safe dependency upgrades**
  - **Source:** Renovate/Dependabot PR
  - **Stimulus:** Patch/minor library update
  - **Artifact:** CI pipeline & canary
  - **Environment:** CI + production canary
  - **Response:** Build, test, canary; auto-rollback on elevated errors
  - **Response Measure :** ≥ **90%** of patch/minor updates pass CI+canary without manual intervention; rollback completes **≤ 10 min**; **≥ 70% unit test coverage** on critical backend modules.
- **M3 - Containerized deployment**

Example: NFR-013 → All system components *System Components* shall be containerized via Docker Compose for consistent deployment on Akamai Cloud.

  - **Source:** Release engineer
  - **Stimulus:** Promote tagged release
  - **Artifact:** Docker images/Compose, Akamai Connected Cloud
  - **Environment**: Staging → production
  - **Response:** Pull, start, health-check, and route traffic

Copyright © 2025 Tony D Barber. All rights reserved.

    - **Response Measure:** All services deploy via Docker/Compose; green health checks ≤ 2 min; zero-downtime rollout with automated rollback on failure.
- **M4 - API discoverability**

Example: NFR-014 → The backend shall automatically generate an OpenAPI 3.0 (Swagger) specification for all REST APIs, accessible at /docs.

- **Source:** Developer / QA
- **Stimulus:** Needs endpoint definitions
- **Artifact:** REST APIs
- **Environment:** Any
- **Response:** Auto-generate and publish OpenAPI 3.0 docs
- **Response Measure:** /docs serves complete, valid OpenAPI for all endpoints in each release.

QAC - 007 - Data Integrity

Example: NFR-008 → The codebase shall follow modular design principles and achieve at least 90 % unit test coverage on critical backend modules.

- **D1 - Waitlist auto-promotion**
    - **Source:** Registrant cancels from full event
    - **Stimulus:** One seat becomes available
    - **Artifact:** Waitlist service & notifications
    - **Environment:** Normal
    - **Response:** Promote next eligible user FIFO; notify; if paid, hold payment until confirm
    - **Response Measure: 100% FIFO** preserved; **0** over-booking incidents in 90 days; full audit trail recorded.
- **D2 - Duplicate registration prevention**
    - **Source:** Same user double-clicks “Register”
    - **Stimulus:** Two requests within 1 second
    - **Artifact:** Registration API & DB constraints
    - **Environment:** Normal and peak
    - **Response:** Idempotency key deduplicates; only one commit succeeds
    - **Response Measure: 0** duplicate rows per user/event; dedupe collision rate **< 0.1%** of requests.

Copyright © 2025 Tony D Barber. All rights reserved.

7. Utility Tree

- The Utility Tree illustrates how high-level quality attributes decompose into measurable quality scenarios that drive architectural decisions.
- Each branch represents a key quality attribute such as Performance, Availability, Security, Usability, and Maintainability refined into concrete scenarios with explicit performance or compliance targets.
- Scenarios marked with (H, H) or (M, H) denote their relative importance and difficulty, helping prioritize architecture design and trade-off decisions in subsequent phases.

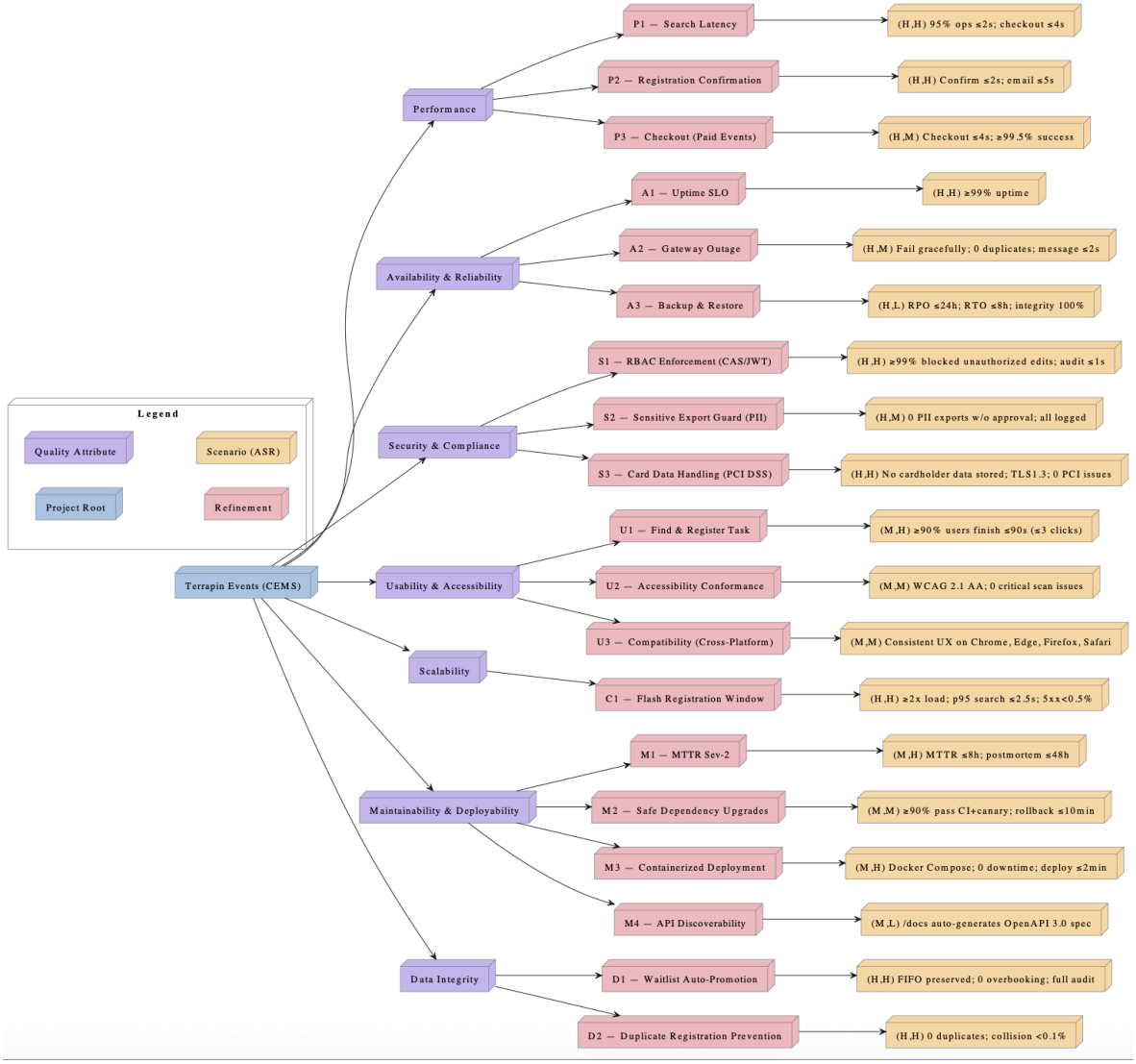


Figure 28 : Utility Tree

Copyright © 2025 Tony D Barber. All rights reserved.

Link to view the diagram : Utility Tree

Copyright © 2025 Tony D Barber. All rights reserved.

Summary of Quality Attributes and Architecturally Significant Requirements (ASRs) (Extra Credit Attempted)

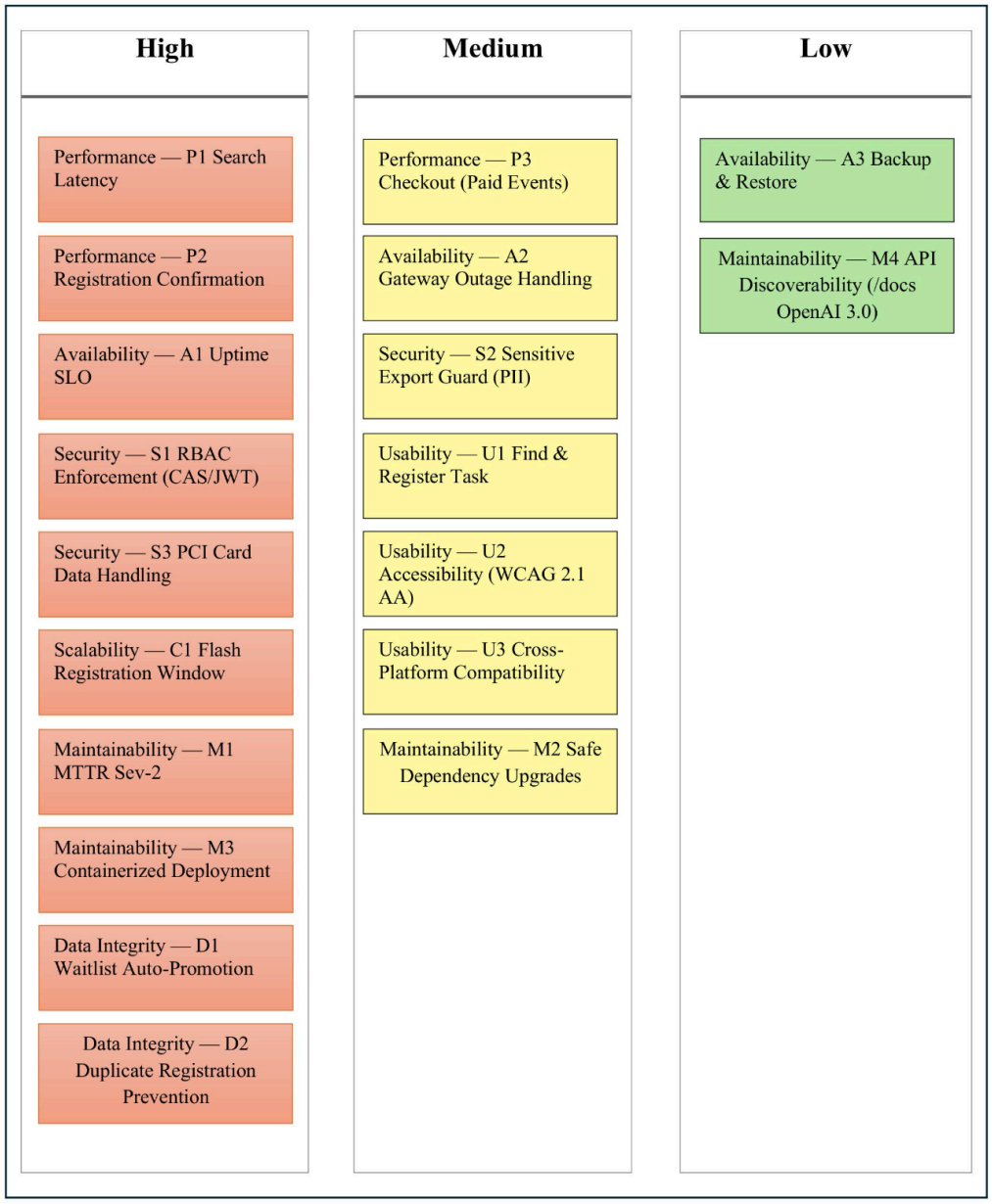


Figure 29: Quality Attribute Scenario Prioritization

Copyright © 2025 Tony D Barber. All rights reserved.

8. System Models

Class Diagram

The Class Diagram of Terrapin Events illustrates the core entities such as Users, Events, Registrations, Payments, and Feedback and their relationships, defining how data and functionality interact across the system.

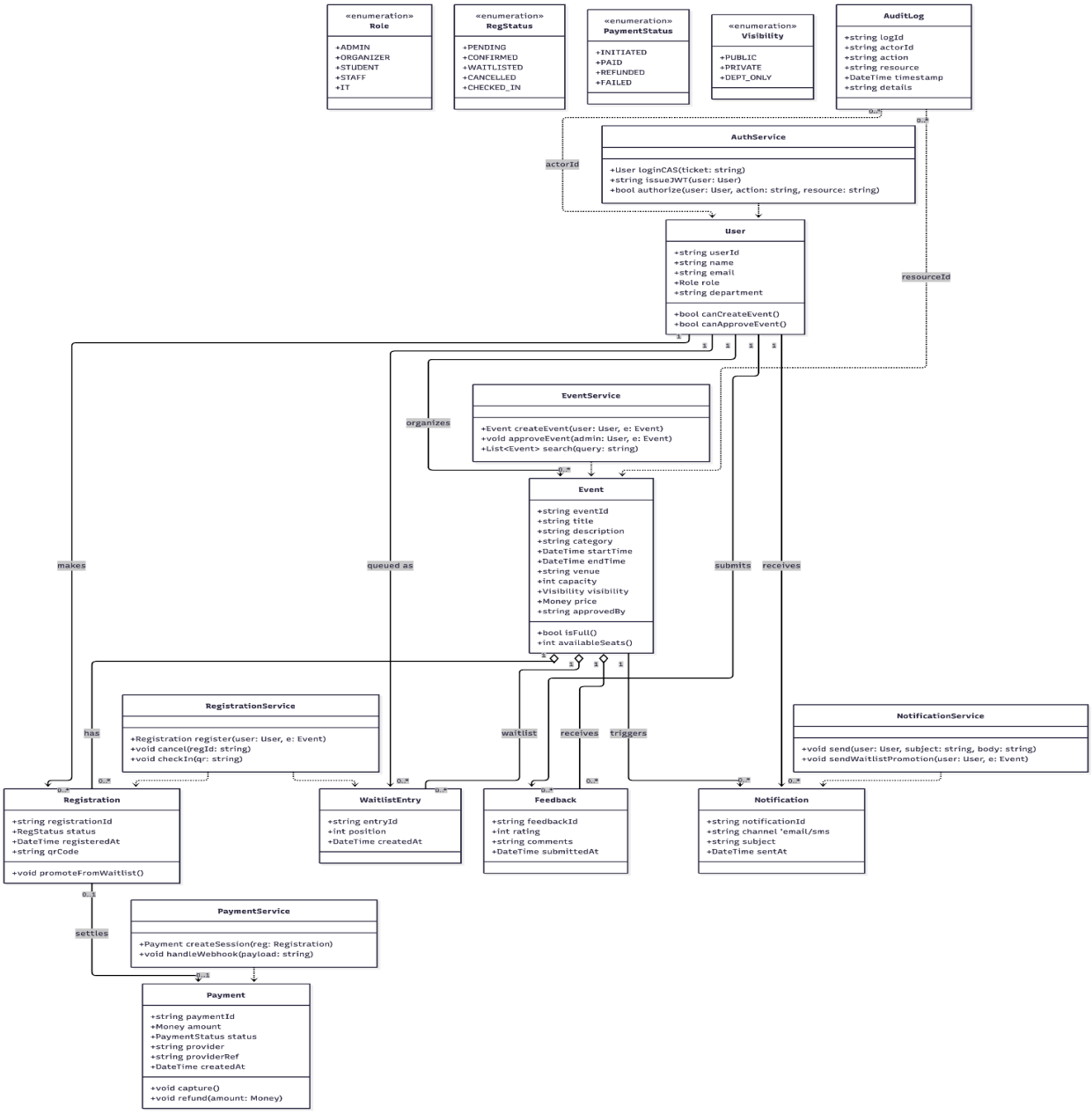


Figure 30: Class Diagram of Terrapin Events

Link to view the diagram : class diagram terrapin events.png

Copyright © 2025 Tony D Barber. All rights reserved.

Sequence Diagram

The Sequence Diagram of Terrapin Events depicts the step-by-step interaction between users and system components such as authentication, event registration, waitlisting, and notifications showing the flow of messages over time.

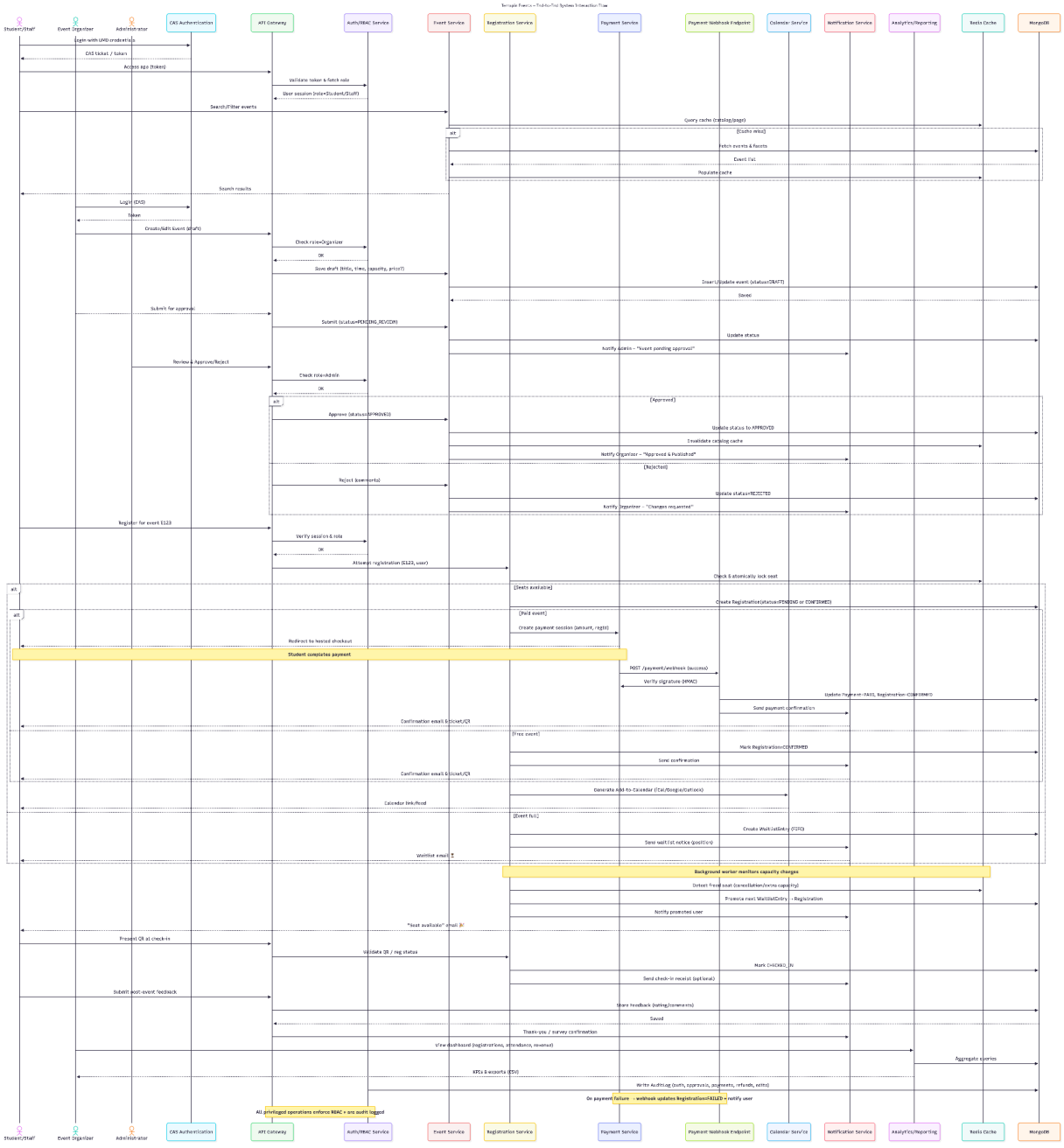


Figure 31: Sequence Diagram for Terrapin Events

Link to view the diagram : sequence diagram terrapin events.png

Copyright © 2025 Tony D Barber. All rights reserved.

Activity Diagrams

The Activity Diagram of Terrapin Events illustrates the dynamic workflow of the system, showing how users progress through key processes like event creation, approval, registration, and feedback submission.

AD-01 UC-01: Sign in with CAS SSO

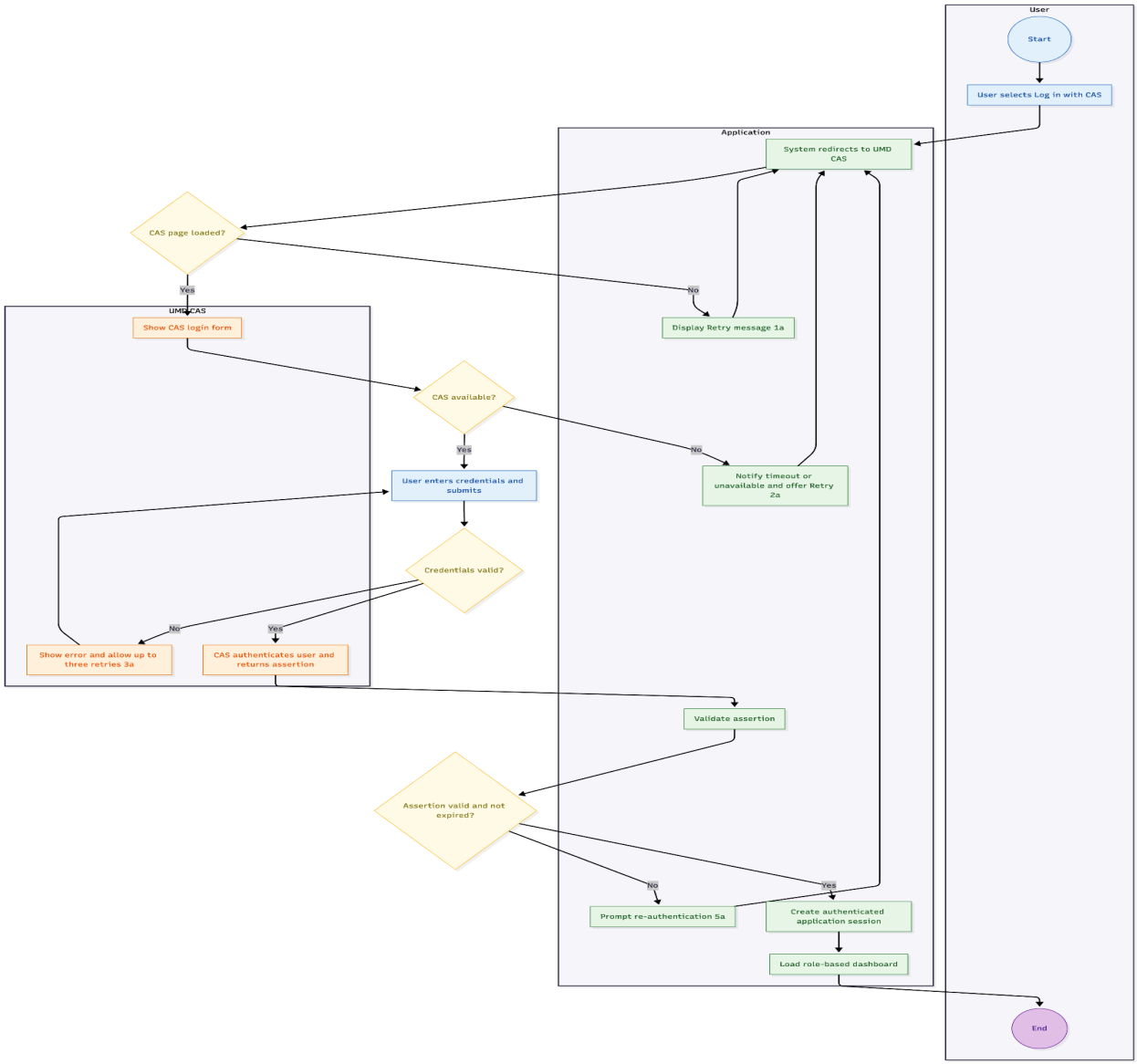


Figure 32: Activity Diagram for Use Case - 001- Sign with CAS SSO

Link to view the diagram : Activity Diagram-001

Copyright © 2025 Tony D Barber. All rights reserved.

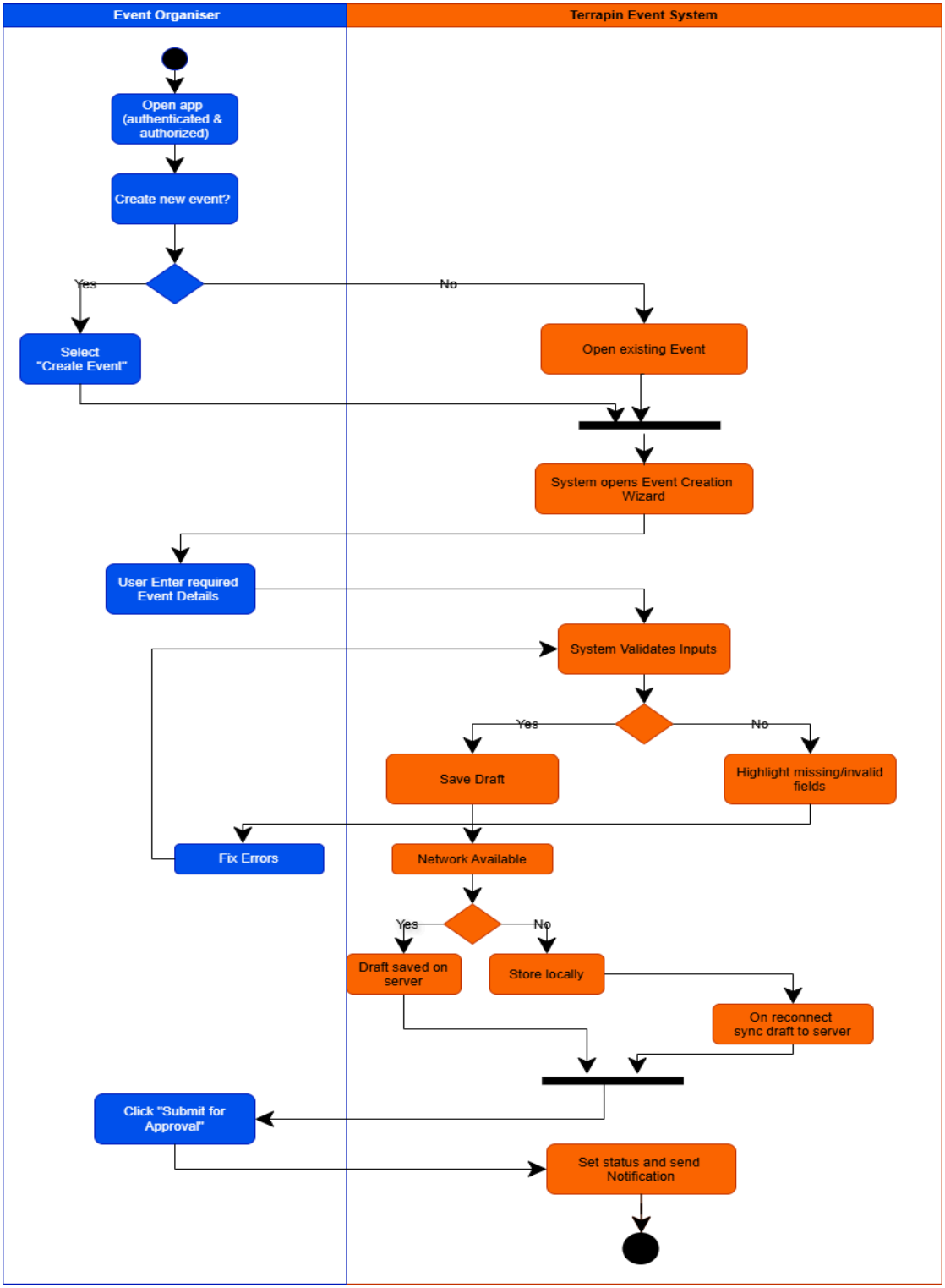
Activity Diagram-01 Legend Description

Type/Style	Meaning
🟦 User (blue boxes)	Actions performed by the authorized user (the actor).
🟩 Application (green boxes)	Steps handled by the application system.
🟧 UMD CAS (orange boxes)	Steps managed by the CAS authentication server.
🟨 Decision (yellow diamonds)	Branch conditions where the process splits based on a Yes/No outcome.
🟪 Final node (purple circle)	End of process indicates the successful completion of authentication.
⚪ Start node (white circle)	Entry point into the activity diagram.
🔁 Arrows	Flow of control showing what happens next depending on conditions.
🧩 Alternate / Exception Flows	Labeled in parentheses (e.g. 1a, 2a, 3a, 5a) to match your SRS use case exception numbering.

Table 8 : Legends Description of AD-001

Copyright © 2025 Tony D Barber. All rights reserved.

AD-02 UC-02: Create and Submit an Event for Approval



Copyright © 2025 Tony D Barber. All rights reserved.

Figure 33: Activity Diagram for Use Case - 002- Create and Submit Event for Approval

Link to view the diagram : Activity Diagram-002

Activity Diagram-02 Legend Description

Type/Style	Meaning
🟦 Event Organiser (blue boxes)	Actions performed by the authorized user (the actor).
🟧 Application (green boxes)	Steps handled by the application system.
Decision (diamonds)	Branch conditions where the process splits based on a Yes/No outcome.
Final node (ending circle)	End of process indicates the successful completion of authentication.
Start node (starting circle)	Entry point into the activity diagram.
🔁 Arrows	Flow of control showing what happens next depending on conditions.
🧩 Alternate / Exception Flows	Labeled in parentheses (e.g. 1a, 2a, 3a, 5a) to match your SRS use case exception numbering.

Table 9 : Legends Description of AD-002

Software Requirement Specification Phase (Extra Credit Attempted)

- This completes the entire Requirements Engineering Process.

Copyright © 2025 Tony D Barber. All rights reserved.

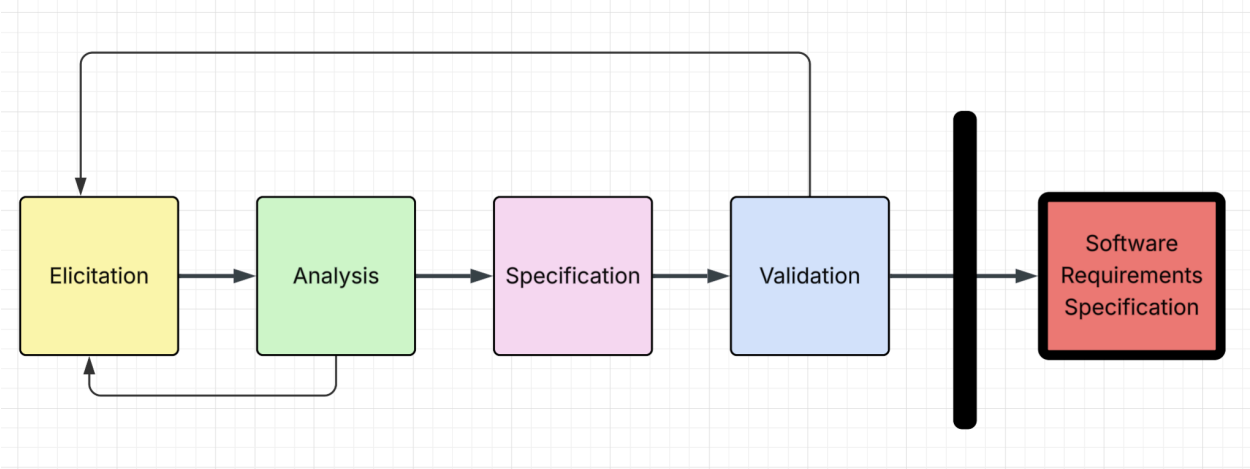


Figure 34: Software Requirement Specification

Agile Update (Extra Credit Attempted)

From October 3rd to October 17th, our Sprint 2, which focused on the Software Requirements Specification (SRS) phase, was successfully completed within the scheduled timeline.

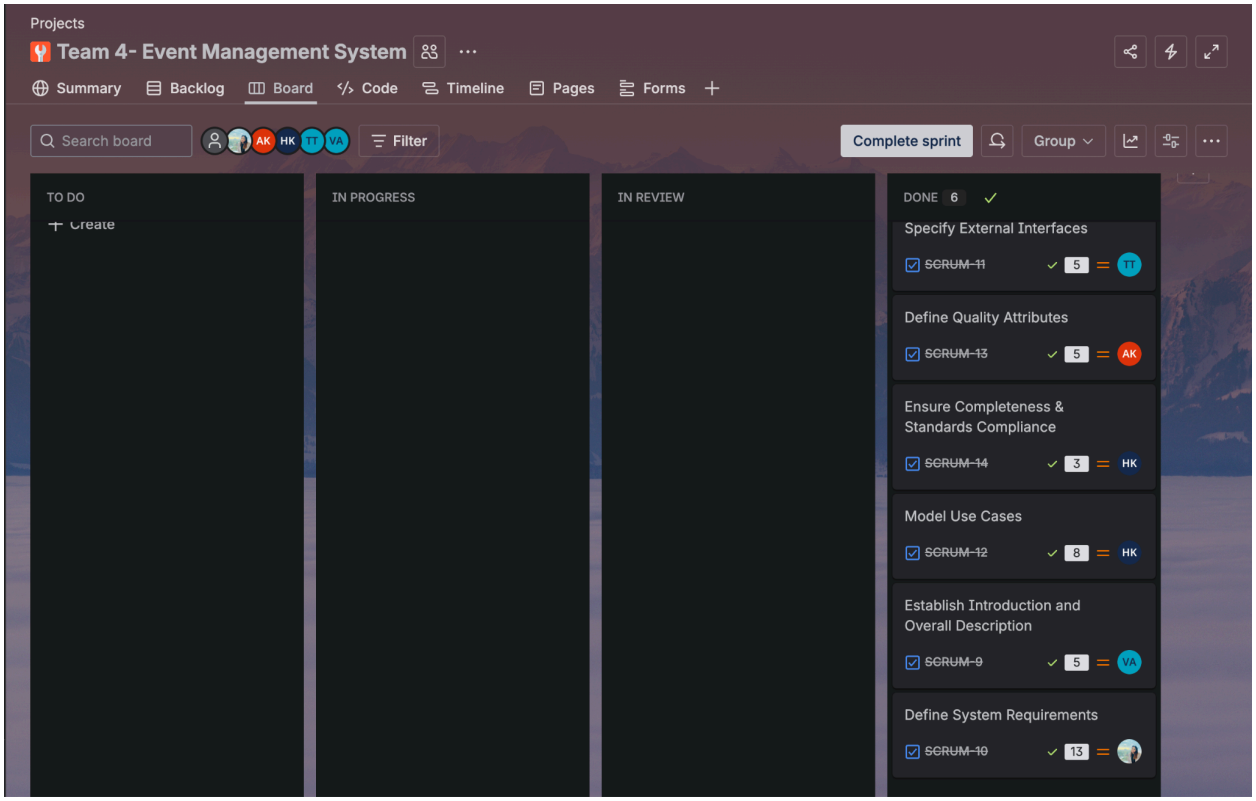


Figure 35: Sprint 2

Copyright © 2025 Tony D Barber. All rights reserved.

Progress Tracker Update (Extra Credit Attempted)

ENPM613-Team4-PMP-Progress Tracker

		Effort (Person hours)											
MAIN ACTIVITY (Use the WBS to Update)	Sub-tasks	Heena Khan (Project Manager)		Bhavna Kumari (Requirements Engineer)		Teja Tatimatla (Solutions Architect)		Vineet Agarwal (Developer)		Abhigna Kandala (Developer)		TOTAL ESTIMATED EFFORT per activity	TOTAL ACTUAL EFFORT per activity
	Sub-tasks	Estimated	Actual	Estimated	Actual	Estimated	Actual	Estimated	Actual	Estimated	Actual		
Project Management Plan	Draft PMP (Structure, purpose, Context)			8	7							8	7
	Create Work Breakdown Strcuture (WBS)							8	7			8	7
	Develop RACI Matrix	6	6									6	6
	Risk Register Preparation					8	8					8	8
	Preparation of Figures									8	8	8	8
	Progress Tracker SetUp	2	2									2	2
	Review & Finalize PMP Document	2	2	2	2	2	2	2	2	2	2	10	10
Requirements	Conduct stakeholder interviews & surveys	2	2	4	4	2	2	2	2	2	2	12	12
	Document functional requirements	2	2	2	2	2	2	2	2	2	2	10	10
	Document non-functional requirements and design constraints	2	2	2	2	2	2	2	2	2	2	10	10
	Build requirements traceability matrix (RTM)	0	0	6	5	0	0	0	0	0	0	6	5
	Validate requirements with stakeholders	2	2	2	2	2	2	2	2	2	2	10	10

Figure 36: Progress Tracker

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix A

References

- Grading Rubric provided by the professor.
- Software Requirements Specification slides provided by the professor.
- Software Requirements Specification template provided by the professor.
- Jira by Atlassian.
- Software Development Lifecycle image taken from link.
- Types of Non Functional requirements image from link.
- Abuse case , Use case diagram and Feature Tree made from plantuml.
- Chart diagrams made on Lucid chart link
- Activity Diagrams using draw.io

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix B

Acronyms:

- **UMD CAS (Central Authentication Service) -** A single sign-on protocol used by the University of Maryland to authenticate students, faculty, and staff. All users must have a UMD Directory ID to access the system.
- **UID -** University of Maryland Identification Number
- **RTM(Requirements Traceability Matrix) -** A document mapping each requirement to its corresponding design, implementation, and test cases to ensure full coverage.
- **SRS(Software Requirements Specification) -** A Software Requirements Specification (SRS) defines the complete behavior, features, and constraints of a software system. It serves as a blueprint that aligns stakeholders, developers, and testers on what the system will deliver.
- **PAN -** Primary Account Number
- **CVV -** Card Verification Value
- **CTA -** Call To Action
- **ADA -** Americans With Disability Act
- **ARIA -** Accessible Rich Internet Application
- **RBAC -** Role-Based Access Control
- **SSO -** Single Sign On
- **JWT -** JSON Web Tokens
- **PII -** Personally Identifiable Information
- **RPO -** Recovery Point Objective
- **JSON -** JavaScript Object Notation
- **FERPA -** Family Educational Rights and Privacy Act
- **PCI DSS -** Payment Card Industry Data Security Standard. It is a set of security standards designed to ensure that all companies that accept, process, store, or transmit credit card information maintain a secure environment.
- **TLS -** Transport Layer Security. It is a cryptographic protocol that ensures secure communication over a computer network by encrypting data transmitted between clients and servers, protecting it from eavesdropping or tampering.
- **WCAG -** Web Content Accessibility Guidelines. It is a set of international standards developed by the W3C (World Wide Web Consortium) that provide recommendations for making web content more accessible to people with disabilities, including those with visual, auditory, physical, or cognitive impairments.
- **REST -** Representational State Transfer
- **API -** Application Programming Interface
- **SLO** - Service Level Objective. It is a specific, internal goal used by organizations to ensure they are meeting the broader promises made to customers in a Service Level Agreement (SLA).

Copyright © 2025 Tony D Barber. All rights reserved.

Appendix C

Definitions:

- **Hero Banner** - A large, prominent image, video, or design element at the top of a webpage that serves as the first impression and introduces the site's content.
- **Multi-step Wizard** - A user interface design pattern that breaks down a complex task, like a long form, into a series of smaller, sequential steps
- **Hamburger Menu -** A hamburger menu is a user interface icon with three horizontal lines that, when clicked or tapped, expands to reveal a hidden navigation menu or options.
- **Hamburger Menu Options -** Below are the items included in this menu
    - Registered Events
    - Calender
    - Previously Attended Events
    - Logout
- **One Click Actions -** Below is the list of the one click actions that the Authorised Users can perform:
    - Register
    - Join Waitlist
    - Add to Calendar
    - Share Event
- **Profile Information -** Profile information should contain the following information:
    - FName
    - LName
    - Email
    - Phone Number
    - School Name
- **Event Details -** Event details information should contain the following information:
    - title, description, category, venue, capacity, visibility, and optional ticket price
    - Description
    - Category
    - Venue
    - Capacity
    - Visibility
    - Ticket Price - *optional*
    - Date
    - Location
- **User Initiated Operations -** The following are the operations that the user can initiate:
    - Login
    - Search
    - Registration
- **Authorised Roles -** The following are the authorised roles that the people using the application would have :
    - **Administrator:** Manages user roles and approves/rejects events.

Copyright © 2025 Tony D Barber. All rights reserved.

    - **Event Organizer:** Creates, edits, and manages events; handles registrations and communications for their events.
    - **Student:** Browses and registers for events, joins waitlists, makes payments (if applicable), and submits feedback.
    - **Staff:** Assists in managing departmental or university events, tracks attendance, and supports administrative reporting.
    - **IT Team:** Monitors analytics, and maintains system integrity.
- **Authorised Users -** All the users having valid CAS authentication.
- **Event Driven Requirement -** Requirements are initiated when and only when a triggering event occurs or is detected.
    - The trigger must be something that the system itself can detect.
    - This can help in clarifying or identifying system boundaries.
- **Unwanted Behaviour Requirement -** Circumstances in which the requirement can be invoked (preconditions) or the initiating event (trigger)
    - The expected system behavior (response)
- **Calendar Types -** Below are the calendar types supported by our application :
    - iCal
    - Google Calender
    - Outlook
- **Event Related Notifications -** Event-related notifications are automated messages sent by the system to inform users about important updates or actions related to an event, such as below
    - Registrations
    - Reminders
    - Cancellations
    - Waitlist Promotions
- **Event Insights -** Event Insights refer to the collection and analysis of data such as below to evaluate the success and effectiveness of events :
    - participation levels
    - registration trends
    - feedback statistics.
- **System Functionalities -** Key sensitive actions that define how the system operates and supports user interactions, such as below :
    - User authentication
    - Event creation
    - Approval
    - Payments
    - Refunds
- **Application Monitoring -** It refers to the continuous observation and analysis of a system's uptime, performance, and errors to detect issues and maintain optimal functionality.
- **Role-Based Dashboard Access -** The following are the role based dashboard access that the people using the application would have :
    - **Administrators:** System analytics, pending approvals, and risk summaries.
    - **Event Organizers:** Upcoming events, registrations, and feedback overview.
    - **Students/Staff:** Registered events, calendar view, and recommendations.

Copyright © 2025 Tony D Barber. All rights reserved.

- **Horizontal Scaling -** Horizontal scaling is the process of increasing system capacity by adding more machines or instances to distribute the workload across multiple servers.
- **User Engagement Processes -** User Engagement Processes refers to the key actions users perform on the platform, including below:
    - Event browsing
    - Registration
    - Submitting feedback.
- **Supported Browsers -** Below are the browsers the application would be supporting
    - Google Chrome
    - Safari
    - Microsoft Edge
    - Firefox
- **Compatible Devices -** Below are the compatible devices that the application would be work:
    - Mobile
    - Tablet
    - Desktop
- **System Components -** Below are the system components that the application has:
    - Frontend
    - Backend
    - Database
- **Tracking Information -** It refers to recorded details written below that help trace user actions and system events for accountability and security purposes.
    - Timestamps
    - User Ids

Copyright © 2025 Tony D Barber. All rights reserved.